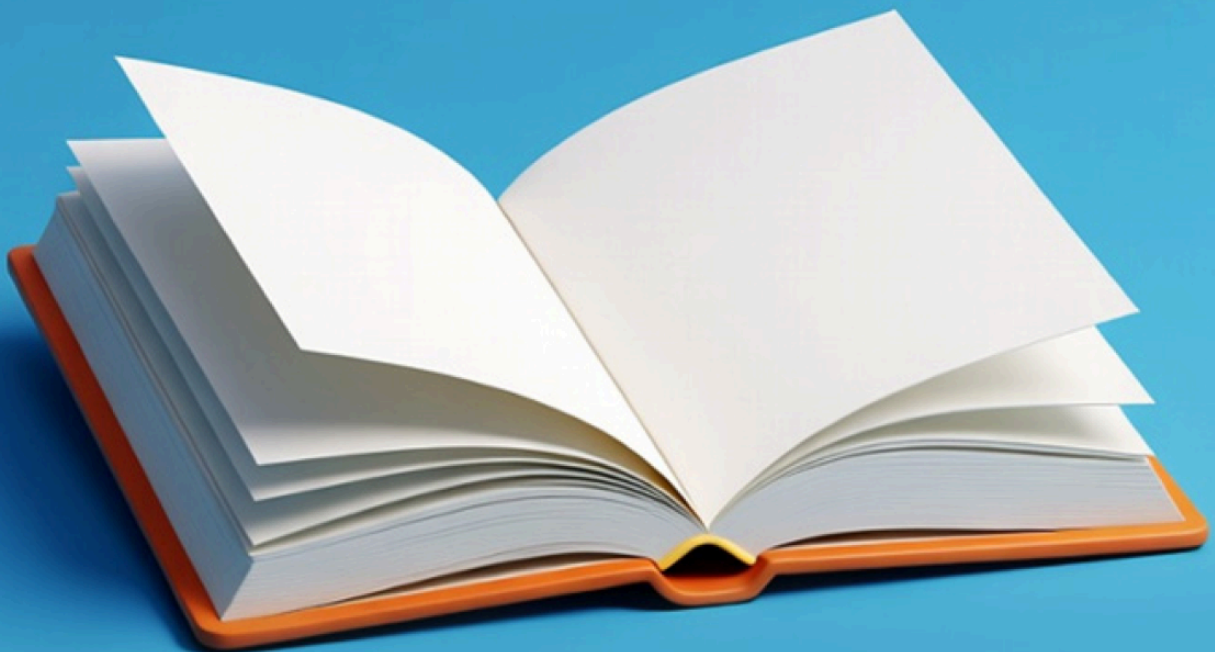


DOSSIER PROJET

# P'tit Cahier

Le cahier de liaison — réinventé, simplifié, centralisé



Yuliana Krasulya

DWWM — septembre 2025 — février 2026  
React · TypeScript · Node.js · Express · MySQL

## Sommaire

|                                                                                                                                                                                                                                 |           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>01 Introduction</b><br>Contexte et problématique · Le projet P'tit Cahier<br>Ma motivation personnelle · Mon apport au projet                                                                                                | <b>3</b>  |
| <b>02 Liste des compétences mises en œuvre</b><br>Activité 1 — Développer la partie front-end<br>Activité 2 — Développer la partie back-end<br>Compétences transversales                                                        | <b>5</b>  |
| <b>03 Expression des besoins</b><br>Public cible · Objectifs fonctionnels · Tableau synthétique des fonctionnalités<br>Contraintes et périmètre · Perspectives d'évolution                                                      | <b>7</b>  |
| <b>04 Environnement technique</b><br>Stack technique · Architecture du projet · Modélisation de la base de données<br>Routes de l'application · Sécurité de l'application<br>Outils de développement et de production           | <b>9</b>  |
| <b>05 Gestion de projet</b><br>Composition de l'équipe · Organisation du travail · Outils de suivi                                                                                                                              | <b>16</b> |
| <b>06 Réalisations personnelles</b><br>Barre de navigation et architecture des layouts (US14)<br>Affichage et gestion des tickets côté école (US02)<br>Modale de détail d'un ticket (US07)<br>Déploiement en production sur VPS | <b>18</b> |
| <b>07 Sécurité de l'application</b><br>Hachage des mots de passe · Authentification par JWT<br>Contrôle des rôles · Validation des données<br>Protection contre les injections SQL                                              | <b>31</b> |
| <b>08 Jeu d'essai</b><br>Scénarios de test de l'authentification · Tableau récapitulatif                                                                                                                                        | <b>38</b> |
| <b>09 Conclusion</b><br>Compétences acquises · Retour d'expérience · Perspectives d'évolution                                                                                                                                   | <b>42</b> |
| <b>Annexes</b>                                                                                                                                                                                                                  | <b>44</b> |

# 01 Introduction

## Contexte et problématique

Imaginez une école où aucune information ne se perd dans un cartable. Imaginez une école sans malentendus entre parents et enseignants... C'est le **P'tit Cahier**.

Dans la majorité des écoles primaires en France, la communication entre l'établissement scolaire et les parents d'élèves repose encore sur des moyens traditionnels. L'un des principaux outils utilisés est le cahier de liaison — un carnet papier par lequel l'école transmet aux familles les informations importantes concernant la vie scolaire.

Les enseignants y inscrivent des messages, des notifications relatives aux événements ou des documents administratifs. L'élève est chargé de transporter ce cahier entre l'école et la maison, et les parents prennent connaissance de son contenu, confirmant parfois la réception par une signature.

Cependant, ce mode de communication présente plusieurs limites. L'information étant transmise par l'intermédiaire de l'enfant, il existe un risque réel que le message ne parvienne jamais à son destinataire : l'élève peut oublier de remettre le cahier, perdre un document ou simplement ne pas en parler à ses parents. Les documents papier peuvent également être endommagés ou égarés.

Par ailleurs, l'école ne dispose d'aucun moyen fiable de vérifier que les parents ont effectivement reçu et lu l'information transmise. Il arrive fréquemment que les messages soient diffusés par plusieurs canaux simultanément — documents papier, courriels, messageries instantanées, annonces orales à la sortie de l'école — ce qui rend la communication fragmentée et augmente le risque de malentendus ou d'oublis. Ce projet s'inscrit dans une logique de résolution d'un problème concret de communication, avec une approche orientée utilisateur et une volonté de proposer un outil réellement utilisable en contexte scolaire.

Considérons deux situations typiques.

### Grève de la cantine

Un jour de grève de la cantine, Camille Martin arrive à l'école sans repas. Certaines familles avaient été informées en amont, mais sa mère n'a pas reçu l'information. L'enfant passe ainsi la journée sans déjeuner. Cette situation illustre le manque de fiabilité des canaux de communication traditionnels et l'absence de diffusion uniforme de l'information.

### Autorisation de sortie scolaire

Dans le cadre d'une sortie scolaire en Allemagne, une autorisation parentale signée était requise. Lucas Dupont a reçu le document, mais ne l'a pas transmis à ses parents. Informés trop tard, ces derniers n'ont pas pu signer l'autorisation à temps, empêchant Lucas de participer au voyage. Ce cas met en évidence les limites d'une communication reposant sur l'intermédiaire de l'élève.

Dans ces deux situations, le problème n'est pas lié à un manque de bonne volonté de la part des parents ou des élèves, mais à l'absence d'un canal de communication fiable, centralisé et indépendant de l'enfant.

## Le projet P'tit Cahier

Le P'tit Cahier est une application web full-stack développée dans le cadre du Titre Professionnel Développeur Web et Web Mobile (DWWM), au sein d'une équipe de quatre personnes à la Wild Code School. Le développement s'est déroulé sur huit semaines, de la conception fonctionnelle à la mise en production sur un serveur dédié. P'tit Cahier est une plateforme de communication entre l'école primaire et les familles. Elle répond à un constat simple : une partie des informations scolaires se perd entre l'école et le domicile. L'application centralise les échanges dans un espace unique, accessible à tout moment depuis un ordinateur ou un téléphone.

Résultats obtenus: l'application full-stack déployée et accessible en ligne à l'adresse <https://ptit-cahier.fr>. Les fonctionnalités livrées, les contraintes du projet et les perspectives d'évolution sont détaillées au chapitre 03.

Le projet repose sur deux axes majeurs :

**Une interface utilisateur claire et accessible**, développée avec React 19 et TypeScript, offrant une navigation intuitive, des filtres par catégorie, enfant et statut, des tableaux de bord personnalisés, ainsi qu'une attention particulière à l'accessibilité numérique (lisibilité typographique, contrastes, attributs ARIA, navigation complète au clavier).

**Une architecture serveur structurée et sécurisée**, bâtie avec Node.js, Express et TypeScript, reposant sur une base de données MySQL. L'authentification JWT, le hachage des mots de passe avec Argon2id, la validation des données avec Joi, la gestion des rôles par middleware et les requêtes SQL paramétrées assurent la fiabilité et la sécurité des données.

## Ma motivation personnelle et difficultés rencontrées

Ma principale difficulté au démarrage du projet n'était pas technique. Elle était liée au sujet lui-même. Le thème de l'application — la communication scolaire — a été choisi par la majorité de l'équipe et le formateur. Je n'ai pas d'enfants, et je n'avais aucune connaissance concrète des problèmes quotidiens auxquels sont confrontés les enseignants et les parents d'élèves. Cahier de liaison, menus de cantine, grève du personnel municipal, autorisations de sortie — tout cela m'était parfaitement étranger. J'ai cependant choisi d'aborder cette situation comme un défi professionnel. Dans la réalité du métier de développeur, on ne choisit pas la thématique d'un projet — on travaille sur la mission qui se présente, quelle que soit sa proximité avec notre vécu personnel. C'est exactement l'approche que j'ai adoptée : j'ai écouté attentivement mes collègues qui avaient des enfants scolarisés, et j'ai interrogé des amis parents d'élèves pour comprendre les difficultés concrètes qu'un bon outil de communication pourrait résoudre. Leurs témoignages m'ont permis de m'approprier un domaine métier que je découvrais entièrement. Cette expérience m'a appris que la compréhension des besoins utilisateurs n'est pas une connaissance innée, mais une compétence qui se construit par l'observation, le questionnement et l'écoute. Le déploiement, enfin, a constitué un véritable parcours d'apprentissage autonome. Chaque erreur de configuration — Nginx, PM2, MySQL — m'amenait à consulter la documentation, analyser les logs et

expérimenter des solutions. Cette étape, la plus exigeante du projet, m'a permis de développer une réelle autonomie face à des problématiques techniques que la formation n'avait pas abordées.

## Mon apport au projet

Au sein de l'équipe, j'ai pris en charge le fil de tickets côté école (US02), la modale de détail avec changement de statut (US07) et la barre de navigation avec les layouts différenciés par rôle (US14). J'ai également assuré l'adaptation CSS globale de l'application, le remplacement des données fictives par du contenu réaliste, l'intégration de mesures d'accessibilité numérique, et l'intégralité du déploiement en production. Ces réalisations sont détaillées au chapitre 06.

Enfin, j'ai introduit dans le projet une dimension qui ne figurait pas dans le cahier des charges initial : l'accessibilité numérique. Dès la phase de conception des maquettes, j'ai orienté les choix typographiques et chromatiques vers une lisibilité optimale — sélection de polices adaptées, vérification des niveaux de contraste, hiérarchie visuelle claire. En fin de projet, j'ai complété cette démarche par l'implémentation d'une navigation intégralement opérable au clavier sur la version desktop, ainsi que par l'ajout d'attributs ARIA sur les éléments interactifs.

---

## 02 Liste des compétences mises en œuvre

Le projet P'tit Cahier couvre l'ensemble des compétences du référentiel du Titre Professionnel Développeur Web et Web Mobile, réparties entre les deux activités types du diplôme.

### Activité 1 — Développer la partie front-end d'une application web ou web mobile sécurisée

**Maquetter une application.** Les maquettes de l'application ont été conçues dans Figma, en deux étapes successives : d'abord des wireframes basse fidélité définissant la structure et la navigation de chaque écran (versions desktop et mobile), puis des maquettes haute fidélité intégrant la charte graphique — palette de couleurs différenciée par rôle (dégradé pastel pour les parents, tons bleu-gris pour l'école), typographie (Nunito Bold pour les titres, Source Sans 3 pour le texte courant), composants visuels (cartes, boutons, filtres, navbar) et états d'interface (survol, sélection, erreur).

**Réaliser une interface utilisateur web statique et adaptable.** L'ensemble de l'interface a été développé en React avec TypeScript et stylisé à l'aide de CSS Modules. L'application est intégralement responsive : la mise en page s'adapte à toutes les tailles d'écran grâce à des media queries, des grilles CSS flexibles et une typographie fluide utilisant la fonction `clamp()`. Un reset CSS complet assure la cohérence du rendu entre les navigateurs.

**Développer une interface utilisateur web dynamique.** Les pages de l'application exploitent les hooks React (`useState`, `useEffect`, `useCallback`, `useMemo`) pour gérer l'état local, les effets de bord et l'optimisation du rendu. Le routage côté client est assuré par React Router DOM v7, avec des routes imbriquées et des layouts protégés par rôle. Les interactions dynamiques incluent des filtres combinés par catégorie, enfant et statut, un carrousel d'annonces avec autoplay, des modales avec gestion du focus

et du scroll, ainsi qu'une mise à jour immédiate de l'interface après chaque action utilisateur — sans rechargement de page.

**Communiquer avec un serveur via des requêtes HTTP.** Le frontend communique exclusivement avec le backend via l'API native `fetch`, en envoyant des requêtes REST (GET, POST, PUT, PATCH, DELETE) authentifiées par un token JWT transmis dans l'en-tête `Authorization: Bearer`. Les réponses JSON sont typées avec TypeScript, et les erreurs réseau sont gérées systématiquement.

## Activité 2 — Développer la partie back-end d'une application web ou web mobile sécurisée

**Créer une base de données.** La base de données relationnelle MySQL a été modélisée et créée à partir d'un schéma SQL versionné dans le dépôt Git. Elle comprend dix tables (user, school, parent, student, classroom, announcement, ticket, announcement\_category, ticket\_category) et deux tables de liaison (announcement\_student, ticket\_student), avec des contraintes d'intégrité référentielle (clés étrangères, ON DELETE CASCADE et ON DELETE SET NULL), des index et des types appropriés.

**Développer les composants d'accès aux données.** L'accès à la base de données est assuré par des classes Repository typées en TypeScript, utilisant le driver `mysql2/promise` avec des requêtes SQL paramétrées (placeholders `?`). Chaque entité métier dispose de son propre repository (announcementRepository, ticketRepository, studentRepository, parentRepository, etc.), encapsulant les opérations CRUD et les requêtes complexes avec jointures multiples et agrégations (GROUP\_CONCAT).

**Développer des composants métier.** La logique métier est implémentée dans des contrôleurs (Actions) organisés par module, suivant une architecture inspirée du pattern MVC. Les middlewares Express assurent la validation des données entrantes avec Joi, la vérification de l'authentification JWT, le contrôle des rôles (parent/school), et la vérification des règles métier (appartenance d'un élève à un parent ou à une école). Le routeur Express sépare les routes par rôle à l'aide de sous-routeurs dédiés (parentRouter, schoolRouter), chacun protégé par un middleware de vérification de rôle.

**Déployer une application web.** L'application a été déployée manuellement sur un serveur VPS OVHcloud sous Ubuntu 24.04 LTS, avec configuration de Nginx en reverse proxy, gestion du processus Node.js via PM2, accès SSH sécurisé par clé Ed25519 et gestion des variables d'environnement en production. Cette compétence a été entièrement acquise en autonomie.

**Gérer le code source avec Git.** Le projet a été développé en utilisant Git et GitHub, avec une stratégie de branches par fonctionnalité, des pull-requests et des revues de code avant chaque fusion dans la branche principale.

**Prendre en compte l'accessibilité numérique.** L'accessibilité a été intégrée dès la conception des maquettes (choix de polices lisibles, vérification des contrastes) et complétée en fin de projet par l'implémentation d'attributs ARIA sur les éléments interactifs, d'une classe utilitaire `.sr-only` pour les lecteurs d'écran, du respect de la préférence `prefers-reduced-motion`, et d'une navigation complète au clavier sur la version desktop.



## 03 Expression des besoins

### Origine du projet

Le projet P'tit Cahier s'inscrit dans le cadre du projet final (Projet 3) de la formation à la Wild Code School. Il ne repose pas sur un cahier des charges fourni par un commanditaire, mais sur une expression des besoins définie par l'équipe en collaboration avec le formateur. Le thème — une application de communication entre école et parents — a été sélectionné par vote. Le projet a été mené selon une approche agile, avec des besoins fonctionnels formalisés sous forme de user stories au sein d'un product backlog, utilisé comme référentiel tout au long du développement.

#### Public cible

##### L'application s'adresse à deux catégories d'utilisateurs :

**L'école** (direction, équipe administrative, enseignants) utilise la plateforme pour diffuser des informations aux familles, consulter et traiter les demandes des parents, ainsi que gérer les données relatives aux élèves et à leurs responsables légaux. Elle peut notamment créer, modifier et supprimer les profils des élèves et des parents.

**Les parents** d'élèves utilisent l'application pour recevoir les communications de l'école et transmettre des demandes structurées. L'accès à l'information est contextualisé : selon la nature des annonces ou des tickets, les parents peuvent consulter des contenus concernant l'ensemble de l'établissement, une ou plusieurs classes, ou uniquement leurs propres enfants.

#### Problème à résoudre

La communication entre l'école et les familles reste fragmentée et peu fiable. Le cahier de liaison papier, encore utilisé, repose sur l'enfant comme intermédiaire, ce qui entraîne des risques de perte, d'oubli et de non-transmission.

Les outils numériques actuels ne répondent pas pleinement au besoin : les e-mails peuvent être ignorés ou filtrés, et les messageries instantanées (ex. WhatsApp) génèrent un volume d'informations difficile à trier.

Les parents rencontrent ainsi des difficultés à identifier rapidement les informations pertinentes et à suivre la scolarité de leur enfant.

P'tit Cahier propose une solution centralisée, structurée et accessible à tout moment, facilitant l'accès à l'information et réduisant la charge mentale des utilisateurs.

### Objectifs fonctionnels

Les priorités fonctionnelles ont été définies avec le formateur et structurées en deux niveaux.

**V1 — MVP.** L'application devait proposer deux rôles distincts (parent, école), chacun avec un tableau de bord personnalisé. Les flux principaux de communication devaient être opérationnels : consultation des annonces côté parent avec filtrage (US01, US05) et consultation des tickets côté école par ordre chronologique (US02, US06). Les formulaires de création — ticket (US03) et annonce (US04) — constituaient le socle fonctionnel minimal.

**V2 — Enrichissement.** La seconde itération a introduit les fonctionnalités complémentaires : authentification sécurisée par JWT (US11), tableaux de bord enrichis (US08, US09), gestion des tickets avec modale et changement de statut (US07), visualisation des annonces côté école (US10), navigation différenciée par rôle avec layouts dédiés (US14), ainsi que les interfaces de gestion des élèves et des parents. Des fonctionnalités initialement classées en « Extra » ont également été réalisées, notamment la modification et la suppression d'annonces.

**Tableau synthétique des fonctionnalités**

| Fonctionnalité                                                            | Rôle   | Version | Statut      |
|---------------------------------------------------------------------------|--------|---------|-------------|
| Fil d'annonces avec filtres par catégorie et par enfant                   | Parent | V1      | Livré       |
| Fil de tickets par ordre chronologique                                    | École  | V1      | Livré       |
| Formulaire de création de ticket                                          | Parent | V1      | Livré       |
| Formulaire de création d'annonce avec ciblage par classe/élève            | École  | V1      | Livré       |
| Filtres avancés sur les tickets (catégorie, statut, tri par date)         | École  | V1      | Livré       |
| Authentification sécurisée (JWT, Argon2id)                                | Tous   | V2      | Livré       |
| Tableaux de bord personnalisés (statistiques, carrousel, tickets récents) | Tous   | V2      | Livré       |
| Modale de détail ticket avec changement de statut                         | École  | V2      | Livré       |
| Barre de navigation adaptée par rôle + layouts dédiés                     | Tous   | V2      | Livré       |
| Gestion des annonces (modification, suppression)                          | École  | Extra   | Livré       |
| Gestion des élèves et des parents (CRUD complet)                          | École  | V2      | Livré       |
| Page de profil parent (modification des données personnelles)             | Parent | —       | Non réalisé |
| Agenda personnalisé (calendrier des événements à venir)                   | Parent | —       | Non réalisé |
| Confirmation de lecture des annonces                                      | Tous   | —       | Non réalisé |
| Carte d'identification de l'élève (student card)                          | Tous   | —       | Non réalisé |

## Contraintes et périmètre

**Durée.** Le développement s'est étendu de décembre 2025 à mi-février 2026, soit environ dix semaines.

**Équipe.** Le projet a été initialement dimensionné pour une équipe de six développeurs. En cours de réalisation, deux membres ont quitté la formation, ce qui a conduit l'équipe restante de quatre personnes à absorber un volume de travail conçu pour six. Cette contrainte a nécessité une repriorisation des fonctionnalités et explique que certaines user stories (profil parent, agenda, confirmation de lecture) n'ont pas pu être livrées dans le temps imparti.

**Choix techniques.** Le stack technique a été imposé par le cadre pédagogique de la Wild Code School : monorepo JavaScript/TypeScript avec React côté client, Express côté serveur et MySQL pour la base de données. Le choix de l'hébergement (VPS OVHcloud) et la méthode de déploiement (manuel, sans CI/CD) ont été déterminés par mes soins.



## Perspectives d'évolution

Les fonctionnalités non réalisées constituent la feuille de route naturelle d'une prochaine itération : page de profil permettant aux parents de modifier leurs données personnelles, agenda personnalisé avec calendrier des événements à venir, système de confirmation de lecture des annonces (avec accusé de réception côté école), et carte d'identification visuelle de l'élève sur chaque annonce.

---

## 04 Environnement technique

### Vue d'ensemble

Le projet P'tit Cahier repose sur une architecture full-stack JavaScript/TypeScript, organisée en monorepo. Ce choix de stack a été défini par le cadre pédagogique de la Wild Code School, avec une latitude sur les bibliothèques complémentaires et les outils de déploiement. L'architecture du projet repose sur une séparation nette entre trois couches : **le frontend** (interface utilisateur), **le backend** (logique métier et API) et la **base de données** (persistance). Ces couches communiquent selon un flux classique : les actions de l'utilisateur dans l'interface React déclenchent des requêtes HTTP vers l'API REST Express, qui traite la demande, interroge la base de données MySQL et renvoie une réponse JSON au client. Cette architecture client-serveur, déployée sur un VPS OVHcloud avec Nginx en reverse proxy, est détaillée dans les sections suivantes.

**Frontend — React 19, TypeScript, Vite.** React a été choisi pour sa capacité à construire des interfaces dynamiques à partir de composants réutilisables, avec un système de gestion d'état local via les hooks. TypeScript apporte le typage statique, ce qui réduit les erreurs à l'exécution et améliore la maintenabilité du code — un avantage significatif dans un projet en équipe où plusieurs développeurs interviennent sur les mêmes fichiers. Vite remplace Create-React-App comme outil de build : son démarrage quasi instantané en développement (grâce au Hot Module Replacement) et sa compilation optimisée en production accélèrent considérablement le cycle de développement.

**Stylisation — CSS Modules.** Chaque composant dispose de son propre fichier `.module.css`, ce qui scope automatiquement les classes CSS au composant et élimine les conflits de noms. Ce choix a été préféré à une bibliothèque de composants (type Material UI) afin de garder un contrôle total sur le design et de renforcer notre maîtrise du CSS.

**Routage — React Router DOM v7.** La navigation côté client utilise `createBrowserRouter` avec des routes imbriquées (nested routes) et des layouts protégés par rôle. Ce système permet de gérer l'authentification, la redirection et l'affichage conditionnel sans rechargement de page.

**Icônes — Lucide React.** Les icônes SVG sont importées individuellement depuis Lucide React, ce qui permet au bundler Vite d'éliminer les icônes non utilisées (tree-shaking) et de réduire la taille du bundle final.

**Backend — Node.js 20 LTS, Express.js, TypeScript.** Node.js permet d'utiliser JavaScript côté serveur, assurant une cohérence de langage entre le frontend et le backend. Express.js est un framework minimaliste qui offre un contrôle précis sur le routage, les middlewares et la gestion des erreurs. TypeScript est utilisé également côté serveur, garantissant le typage des requêtes, des réponses et des interactions avec la base de données.

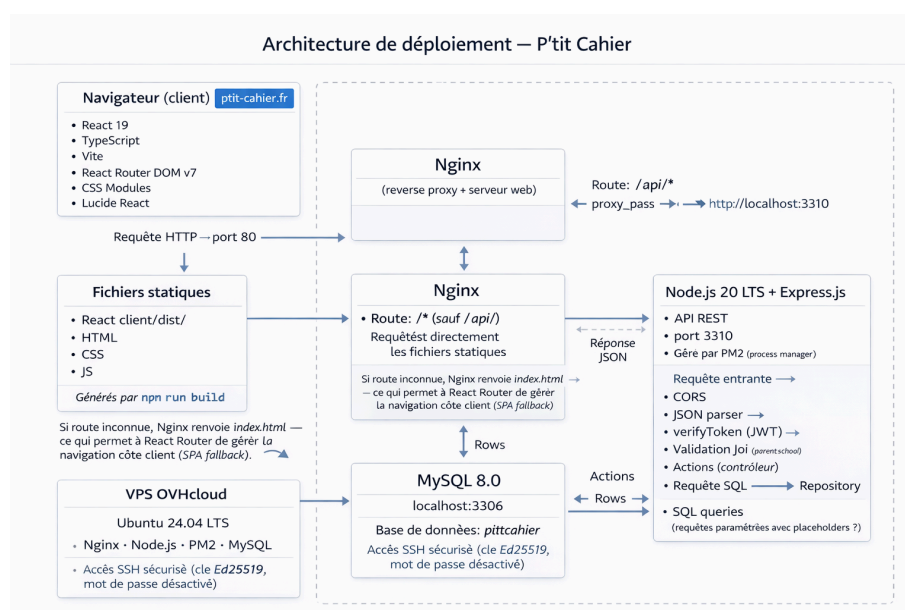
**Base de données — MySQL 8.0, mysql2/promise.** MySQL a été choisi comme système de gestion de base de données relationnelle. L'accès aux données utilise le driver `mysql2/promise` avec des requêtes SQL écrites directement (pas d'ORM). Ce choix pédagogique permet de maîtriser le SQL et de comprendre précisément ce qui est exécuté sur la base, contrairement à un ORM qui abstrait ces opérations.

**Validation — Joi.** La bibliothèque Joi valide la structure et le contenu de chaque requête entrante côté serveur, avant tout traitement. Cela protège l'application contre les données malformées et constitue la première ligne de défense contre les entrées malveillantes. Concrètement, la validation est implémentée sous forme de middleware Express, exécuté dans la chaîne de routage avant le contrôleur : la requête passe d'abord par `verifyToken`, puis `verifyRole`, puis le middleware de validation Joi, et seulement si toutes ces étapes réussissent, elle atteint le contrôleur (Actions) qui exécute la logique métier.

**Authentification — JSON Web Tokens (jsonwebtoken), Argon2.** L'authentification repose sur des tokens JWT signés avec une clé secrète, transmis dans l'en-tête HTTP `Authorization: Bearer`. Les mots de passe sont hachés avec Argon2id, l'algorithme recommandé par l'OWASP et vainqueur du Password Hashing Competition.

**Qualité de code — Biome.** Biome remplace ESLint et Prettier en un seul outil : il assure le linting et le formatage du code selon des règles configurées pour l'ensemble du monorepo, garantissant un style cohérent entre les quatre développeurs.

## Architecture du projet



**Le client React** (compilé en fichiers statiques par Vite) est servi par Nginx. Il envoie des requêtes HTTP vers l'API backend.

**Le serveur Express** reçoit les requêtes, les fait passer par une chaîne de middlewares (CORS, parsing JSON, vérification JWT, vérification de rôle, validation Joi), puis les transmet au contrôleur approprié (Actions), qui appelle le repository correspondant.

**Le repository** exécute les requêtes SQL sur la base MySQL et retourne les données au contrôleur, qui les renvoie au client en JSON.

Côté serveur, le code suit une architecture modulaire inspirée du pattern MVC :

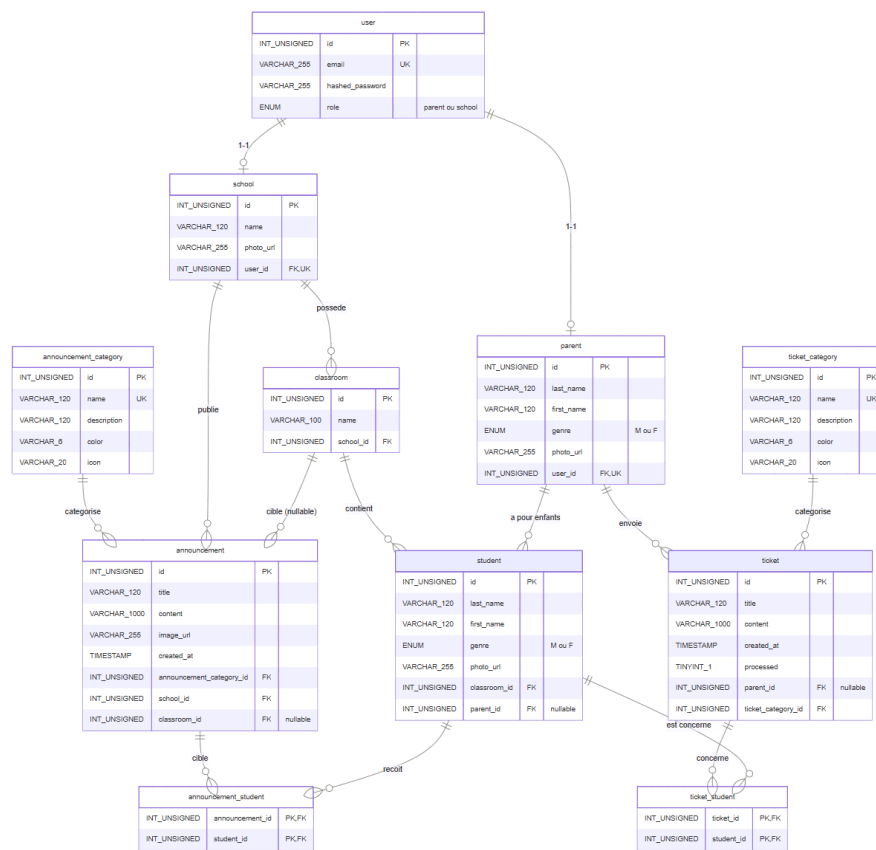
```
server/src/
├─ modules/
│  ├─ announcement/
│  │  ├─ announcementActions.ts    ← Contrôleur (logique de traitement)
│  │  └─ announcementRepository.ts ← Modèle (accès aux données SQL)
│  ├─ ticket/
│  ├─ student/
│  ├─ parent/
│  ├─ school/
│  ├─ auth/
│  ├─ user/
│  ├─ classroom/
│  ├─ announcementCategory/
│  └─ ticketCategory/
├─ router.ts                       ← Routage et middlewares
├─ app.ts                          ← Configuration Express (CORS, JSON, routes)
└─ main.ts                         ← Point d'entrée du serveur
```

Côté client, l'architecture sépare les pages (liées aux routes) des composants réutilisables :

```
client/src/
├─ pages/                          ← Une page = une route dans le routeur
│  ├─ home/                        ← PublicHome, Home (dashboard)
│  ├─ announcement/ ← Announcements, AnnouncementNew
│  ├─ ticket/                  ← Tickets, TicketNew
│  ├─ login/                   ← Login, Register
│  ├─ student/                 ← StudentsTable
│  ├─ parent/                  ← ParentsTable
│  └─ redirection/ ← Redirection post-connexion
├─ components/                  ← Composants UI réutilisables
│  ├─ NavBar/
│  ├─ AnnouncementCard/
│  ├─ TicketCard/
│  ├─ TicketModal/
│  ├─ AnnouncementForm/
│  ├─ TicketForm/
│  ├─ FilterStudents/
│  └─ ...
├─ layouts/                     ← AuthLayout, ParentLayout, SchoolLayout
├─ types/                       ← Définitions TypeScript des entités métier
└─ styles/                      ← Variables CSS, reset, typographie, styles globaux
```

# Modélisation de la base de données

Modèle Physique de Données — P'tit Cahier — MySQL 8.0 — 11 tables dont 2 tables de liaison — 14 clés étrangères



## Modèle Conceptuel de Données (MCD)

Le modèle conceptuel de données du projet P'tit Cahier repose sur plusieurs entités principales représentant les acteurs du système, la structure scolaire et les flux de communication.

### Entités principales et leurs relations

**user** — entité d'authentification. Chaque utilisateur possède un email unique, un mot de passe haché et un rôle (*parent* ou *school*). Un **user** est associé en 1:1 à un **parent** ou à une **school** selon son rôle.

**school** — représente un établissement scolaire. Une **school** est liée à un **user** et possède plusieurs **classroom** (1,N).

**parent** — représente un parent d'élève. Un **parent** est lié à un **user** et peut être associé à plusieurs **student** (0,N).

**classroom** — représente une classe au sein d'une école (CP, CE1, CE2, CM1, CM2). Une **classroom** appartient à une seule **school** et contient plusieurs **student** (1,N).

**student** — représente un élève. Un **student** appartient à une **classroom** et peut être associé à zéro ou un **parent** (0,1).

**announcement** — représente une annonce publiée par une **school**. Une **announcement** appartient à une seule **announcement\_category** et est adressée à un ou plusieurs **student** via une relation N:N. Le ciblage se fait au niveau des élèves : lorsque l'école sélectionne une classe dans l'interface, l'application résout automatiquement cette sélection en une liste d'élèves individuels.

**ticket** — représente une demande envoyée par un **parent**. Un **ticket** appartient à une seule **ticket\_category** et peut concerner plusieurs **student** (relation N,N).

**announcement\_category** et **ticket\_category** — entités de référence contenant les catégories (nom, description, couleur, icône). Chaque catégorie peut être associée à plusieurs entités correspondantes (0,N).

Les contraintes UNIQUE sont appliquées sur trois colonnes : `user.email` (un seul compte par adresse email), `school.user_id` et `parent.user_id` (un seul profil par compte utilisateur, garantissant la relation 1:1 entre `user` et son profil).

## Routes de l'application

### Routes frontend (React Router)

| Path                                   | Page            | Rôle   | Description                                             |
|----------------------------------------|-----------------|--------|---------------------------------------------------------|
| <code>/</code>                         | PublicHome      | Public | Page d'accueil / landing page                           |
| <code>/login</code>                    | Login           | Public | Connexion (email, mot de passe, sélection du rôle)      |
| <code>/register</code>                 | Register        | Public | Inscription d'une nouvelle école                        |
| <code>/parent/home</code>              | Home            | Parent | Tableau de bord parent (carrousel, tickets, calendrier) |
| <code>/parent/announcements</code>     | Announcements   | Parent | Fil d'annonces avec filtres par catégorie et par enfant |
| <code>/parent/tickets</code>           | Tickets         | Parent | Historique des tickets envoyés                          |
| <code>/parent/tickets/new</code>       | TicketNew       | Parent | Formulaire de création de ticket                        |
| <code>/school/home</code>              | Home            | École  | Tableau de bord école (statistiques, derniers tickets)  |
| <code>/school/announcements</code>     | Announcements   | École  | Liste des annonces publiées (modification, suppression) |
| <code>/school/announcements/new</code> | AnnouncementNew | École  | Formulaire de création d'annonce                        |
| <code>/school/tickets</code>           | Tickets         | École  | Liste des tickets reçus (filtres, statuts, modale)      |
| <code>/school/students</code>          | StudentsTable   | École  | Gestion des élèves (CRUD)                               |
| <code>/school/parents</code>           | ParentsTable    | École  | Gestion des parents (CRUD)                              |

### Routes API (Express)

| Méthode | Endpoint                                  | Middleware(s)          | Description                          |
|---------|-------------------------------------------|------------------------|--------------------------------------|
| POST    | <code>/api/login</code>                   | —                      | Authentification                     |
| POST    | <code>/api/register/school</code>         | validate, hashPassword | Inscription école + création classes |
| GET     | <code>/api/ticket-categories</code>       | verifyToken            | Liste des catégories de tickets      |
| GET     | <code>/api/announcement-categories</code> | verifyToken            | Liste des catégories d'annonces      |

**Routes parent** (protégées par `verifyToken` + `verifyRole("parent")`) :

| Méthode | Endpoint                                   | Description                                                                               |
|---------|--------------------------------------------|-------------------------------------------------------------------------------------------|
| GET     | <code>/api/parents/me/school</code>        | École de rattachement                                                                     |
| GET     | <code>/api/parents/me/announcements</code> | Annonces (filtres: <code>?category</code> , <code>?student</code> , <code>?limit</code> ) |
| GET     | <code>/api/parents/me/students</code>      | Enfants du parent                                                                         |
| GET     | <code>/api/parents/me/tickets</code>       | Tickets envoyés (filtre: <code>?limit</code> )                                            |
| POST    | <code>/api/parents/tickets</code>          | Création d'un ticket                                                                      |

**Routes école** (protégées par `verifyToken` + `verifyRole("school")`) :

| Méthode | Endpoint                                       | Description                                         |
|---------|------------------------------------------------|-----------------------------------------------------|
| GET     | <code>/api/schools/me</code>                   | Données dashboard (statistiques)                    |
| GET     | <code>/api/schools/me/announcements</code>     | Annonces publiées (filtre: <code>?category</code> ) |
| GET     | <code>/api/schools/me/tickets</code>           | Tickets reçus (filtre: <code>?limit</code> )        |
| GET     | <code>/api/schools/me/students</code>          | Liste des élèves                                    |
| GET     | <code>/api/schools/me/parents</code>           | Liste des parents                                   |
| GET     | <code>/api/schools/me/classrooms</code>        | Liste des classes                                   |
| POST    | <code>/api/schools/announcements</code>        | Création d'annonce                                  |
| PUT     | <code>/api/schools/me/announcements/:id</code> | Modification d'annonce                              |
| DELETE  | <code>/api/schools/me/announcements/:id</code> | Suppression d'annonce                               |
| PATCH   | <code>/api/schools/tickets/:id/status</code>   | Changement de statut d'un ticket                    |
| POST    | <code>/api/schools/me/students</code>          | Ajout d'un élève                                    |
| PUT     | <code>/api/schools/me/students/:id</code>      | Modification d'un élève                             |
| DELETE  | <code>/api/schools/me/students/:id</code>      | Suppression d'un élève                              |
| PUT     | <code>/api/schools/me/parents/:id</code>       | Modification d'un parent                            |
| DELETE  | <code>/api/schools/me/parents/:id</code>       | Suppression d'un parent                             |



## Sécurité de l'application

La sécurité a été traitée comme une préoccupation transversale, intégrée à chaque couche de l'application :

**Hachage des mots de passe — Argon2id.** Les mots de passe ne sont jamais stockés en clair. L'algorithme Argon2id, recommandé par l'OWASP, est utilisé avec des paramètres configurés (`memoryCost: 19 Mo`, `timeCost: 2`, `parallelism: 1`) pour résister aux attaques par force brute et aux attaques GPU. Le mot de passe en clair est immédiatement supprimé de l'objet requête après hachage (`req.body.password = undefined`).

**Authentification par JWT.** Après vérification des identifiants, le serveur génère un token JWT signé avec une clé secrète stockée en variable d'environnement. Le token contient l'identifiant du profil et le rôle de l'utilisateur, avec une durée de validité d'une heure. Chaque requête vers une route protégée est vérifiée par le middleware `verifyToken`, qui extrait et valide le token depuis l'en-tête `Authorization: Bearer`.

**Contrôle des rôles.** Le middleware `verifyRole` vérifie que le rôle contenu dans le token correspond à la route demandée. Les routes parent et école sont séparées dans des sous-routeurs dédiés (`parentRouter`, `schoolRouter`), chacun protégé par son propre middleware. Un parent ne peut pas accéder aux routes école, et inversement.

**Vérification d'appartenance.** Au-delà du rôle, les middlewares de validation vérifient que les données manipulées appartiennent bien à l'utilisateur connecté. Par exemple, un parent ne peut créer un ticket que pour ses propres enfants — le serveur vérifie que chaque `studentId` est effectivement rattaché au `parentId` du token. De même, une école ne peut modifier que ses propres annonces.

**Validation des données entrantes — Joi.** Chaque requête POST, PUT et PATCH passe par un middleware de validation Joi qui vérifie la structure, le type et les contraintes des données (longueur maximale, format email, entiers positifs, tableaux non vides). Les données invalides sont rejetées avec un statut 400 avant tout traitement.

**Requêtes SQL paramétrées.** Toutes les requêtes SQL utilisent des placeholders (?) avec des paramètres passés séparément, ce qui empêche toute injection SQL. Aucune concaténation de chaîne n'est utilisée pour construire les requêtes.

**CORS.** La configuration CORS limite les origines autorisées à l'URL du client, définie en variable d'environnement. L'utilisation de `cors()` sans restriction (qui autoriserait toutes les origines) a été explicitement évitée.

**Variables d'environnement.** Les secrets applicatifs (identifiants de base de données, clé JWT, URL du client) sont stockés dans un fichier `.env` non versionné dans le dépôt Git. Un fichier `.env.sample` sans valeurs sensibles est fourni comme modèle.

**Protection en production.** L'application déployée fonctionne en mode démonstration (`DEMO_MODE=true`) avec protection en lecture seule (`READ_ONLY_MODE=true`), empêchant toute modification des données de production lors de la présentation devant le jury.

## Outils de développement et de production

| Outil              | Usage                                                                    |
|--------------------|--------------------------------------------------------------------------|
| Visual Studio Code | Éditeur de code principal                                                |
| Figma              | Conception des wireframes et maquettes UI                                |
| Git / GitHub       | Versionnement du code et collaboration (branches, pull requests, revues) |
| Trello             | Gestion de projet (tableau Kanban, backlog de user stories)              |
| Biome              | Linting et formatage du code (remplace ESLint + Prettier)                |
| npm                | Gestionnaire de paquets et scripts (workspaces monorepo)                 |
| Concurrently       | Exécution simultanée du client et du serveur en développement            |
| Nginx              | Serveur web et reverse proxy en production                               |
| PM2                | Gestionnaire de processus Node.js en production                          |
| SSH (Ed25519)      | Accès sécurisé au serveur de production                                  |

## 05 Gestion de projet

### Composition de l'équipe

Le projet a été réalisé par quatre développeurs en formation à la Wild Code School. Chaque membre était responsable de ses user stories de bout en bout — du composant React à la requête SQL — ce qui a permis à chacun de pratiquer l'ensemble de la stack technique.

**Emeric** — annonces côté parent, tableaux de bord parent et école. **Clarissa** — formulaire de ticket, système d'authentification. **Jennifer** — formulaire d'annonce avec ciblage par classe, filtres tickets côté école. **Yuliana (moi-même)** — tickets côté école (US02), modale de détail ticket (US07), navbar et layouts par rôle (US14), adaptation CSS globale, données réalistes en BDD, déploiement complet en production.

### Organisation du travail

L'équipe a adopté une organisation collaborative inspirée de la méthodologie agile, sans attribution formelle de rôles Scrum. Les décisions relatives aux priorités et aux choix techniques étaient prises collectivement lors de points réguliers. Le développement a été structuré en deux itérations (V1 puis V2), dont le détail est présenté au chapitre 03.

## Outils de suivi

**Trello** — tableau Kanban (À faire, En cours, En revue, Terminé), une carte par user story, attribuée à un membre.

**Git et GitHub** — branches par fonctionnalité, pull requests et revues de code avant chaque fusion.

## Communication

L'équipe travaillait en téléprésentiel : chaque membre suivait la formation depuis son domicile via visioconférence. La communication au quotidien s'effectuait sur le serveur Discord de la Wild Code School, aussi bien pendant les sessions de formation qu'en dehors des heures de cours. Les revues de code sur GitHub complétaient ces échanges par un retour technique écrit sur le travail de chacun.

## Résolution d'un problème technique récurrent

Un bug persistant a accompagné l'équipe pendant toute la durée du développement. Deux fichiers de types TypeScript côté serveur — `Ticket.ts` et `School.ts` — avaient été créés en début de projet avec une minuscule initiale (`ticket.ts`, `school.ts`). Bien que le nommage ait été corrigé rapidement, le problème réapparaissait systématiquement après chaque création de branche, chaque merge et chaque pull. Les imports, écrits avec une majuscule (`import type { Ticket } from "../types/express/Ticket"`), ne correspondaient plus au nom réel du fichier, et la compilation échouait. Chaque membre de l'équipe devait alors renommer manuellement les fichiers concernés avant de pouvoir travailler.

Nous n'avons pas réussi à identifier la cause de ce problème pendant le développement. La raison en était simple : elle relevait d'une différence de comportement entre systèmes d'exploitation, un sujet qui n'avait pas été abordé en formation. C'est lors du déploiement sur le serveur VPS que j'ai compris l'origine du bug. Sur Windows et macOS, le système de fichiers est insensible à la casse : `ticket.ts` et `Ticket.ts` sont considérés comme le même fichier. Lorsqu'on renomme un fichier en changeant uniquement la casse, Git ne détecte pas toujours cette modification et conserve l'ancien nom dans son historique. En revanche, sur Linux — le système du serveur de production — la casse est significative : `ticket.ts` et `Ticket.ts` sont deux fichiers distincts. Les imports référençant `Ticket.ts` ne trouvaient donc pas le fichier `ticket.ts`, et le backend refusait de démarrer.

La solution a consisté à corriger les noms de fichiers directement sur le serveur pour que la casse corresponde exactement à celle utilisée dans les imports. Une fois cette harmonisation effectuée, l'erreur a définitivement disparu.

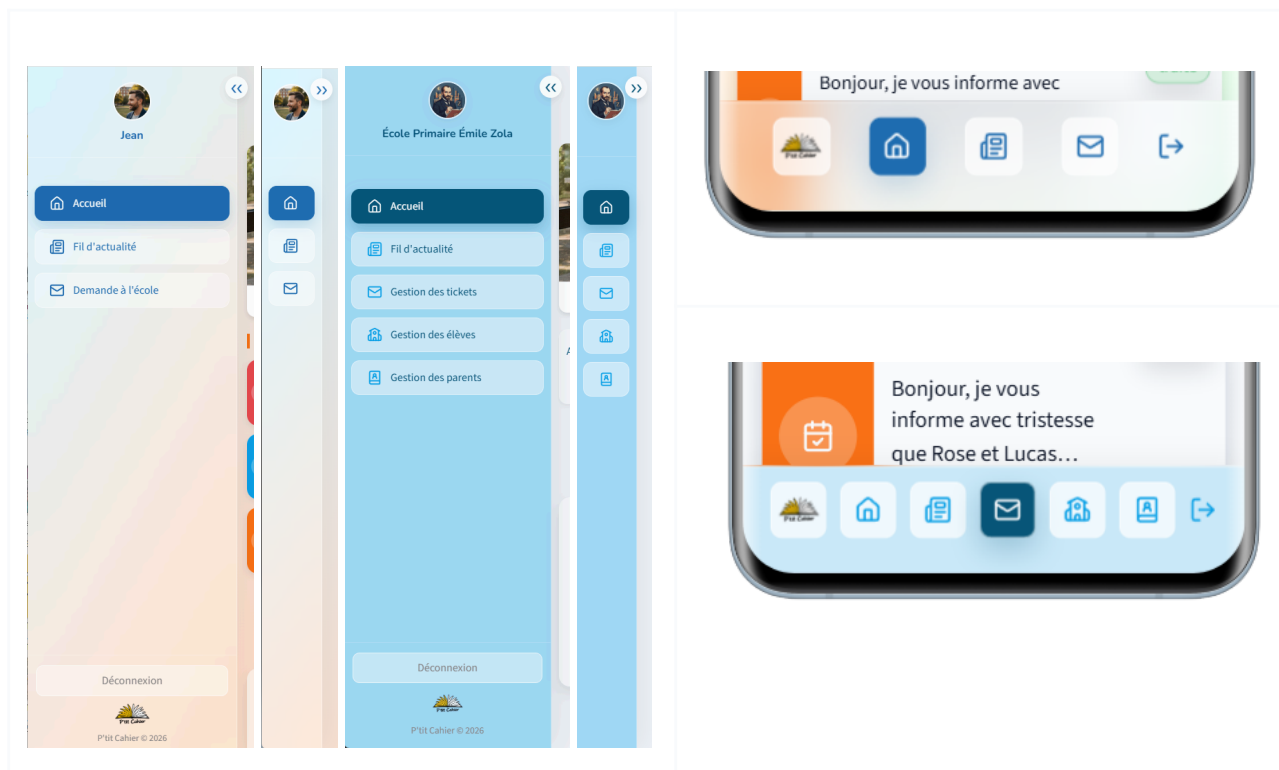
Cette expérience illustre un apprentissage concret : la différence de traitement de la casse entre les systèmes de fichiers (case-insensitive sur Windows/macOS, case-sensitive sur Linux) est un piège classique du déploiement, que seule la confrontation avec un environnement de production réel permet de découvrir.

## 06 Réalisations personnelles

Ce chapitre présente en détail les fonctionnalités que j'ai personnellement conçues et développées dans le cadre du projet P'tit Cahier. Pour chaque réalisation, le parcours complet est décrit : de la maquette à l'interface utilisateur, du composant React à la route API, du middleware de validation à la requête SQL.

### Barre de navigation et architecture des layouts (US14)

**User story :** En tant qu'utilisateur connecté (parent ou école), je souhaite disposer d'une barre de navigation adaptée à mon rôle, afin de me repérer et de naviguer facilement dans l'application.



### Contexte et enjeu

La user story US14 demandait la création d'une barre de navigation permettant aux utilisateurs de se repérer et de naviguer entre les pages de l'application. En apparence, il s'agissait d'un composant d'interface classique. En pratique, cette tâche m'a confrontée à un problème architectural qui dépassait le cadre de ma user story.

L'application P'tit Cahier sert deux types d'utilisateurs — les parents et les écoles — qui n'accèdent pas aux mêmes pages, ne voient pas les mêmes données et ne disposent pas des mêmes actions. Or, au moment où j'ai commencé le développement de la navbar, l'application ne disposait d'aucun mécanisme permettant de séparer ces deux expériences. Toutes les pages étaient rendues au même niveau, sans distinction de rôle.

J'ai proposé à l'équipe la création de deux layouts distincts — ParentLayout et SchoolLayout — chacun encapsulant les pages accessibles par le rôle correspondant, avec sa propre navbar, ses propres styles et sa propre logique de protection des routes. L'équipe a validé cette proposition, et j'ai ensuite adapté l'ensemble des styles CSS de l'application à cette nouvelle architecture.

## Architecture des layouts

La mise en place d'un système de layouts est née d'une problématique concrète rencontrée lors du développement de la navigation. L'application devait proposer deux expériences distinctes selon le rôle (parent et école), avec une structure de page identique mais un contenu, des liens et un style visuel différents.

Initialement, la question portait sur la création de composants de navigation adaptés à ces deux rôles. Très rapidement, il est apparu que dupliquer la logique de rendu de la navigation dans chaque page introduit du code redondant, de la complexité et un risque d'incohérence — toute modification du menu aurait dû être reproduite manuellement dans chaque page.

J'ai donc proposé à l'équipe une architecture basée sur des layouts dédiés (ParentLayout et SchoolLayout), chacun encapsulant sa propre navigation et structurant l'ensemble des pages associées via le système de routing de React Router. Cette approche centralise l'affichage de la navigation à un seul endroit, garantit une séparation nette des responsabilités selon le rôle, et permet d'ajouter de nouvelles pages ou de modifier le menu sans impact sur le reste de l'application.

Le layout devient ainsi un point d'entrée structurant de l'interface, et non un simple composant visuel. L'équipe a validé cette proposition, et j'ai ensuite adapté les styles CSS de toutes les pages existantes à cette nouvelle architecture.

La solution repose sur trois layouts imbriqués dans le routeur React :

**AuthLayout** — le layout racine pour toutes les routes protégées. Il récupère l'état d'authentification depuis le localStorage et le transmet aux composants enfants via useOutletContext.

```
tsx

// client/src/layouts/AuthLayout.tsx
function AuthLayout() {
  const [auth, setAuth] = useState<Auth | null | undefined>(undefined);

  useEffect(() => {
    const storedAuth = localStorage.getItem("auth");
    if (storedAuth) {
      setAuth(JSON.parse(storedAuth));
    } else {
      setAuth(null);
    }
  }, []);

  if (auth === undefined)
    return <div className="general_error_message">Loading...</div>;

  return <Outlet context={{ auth, setAuth }} />;
}
```

**ParentLayout** — vérifie que l'utilisateur connecté a le rôle "parent". Si l'utilisateur n'est pas authentifié, il est redirigé vers /login. Si son rôle ne correspond pas, il est redirigé vers /redirection. Dans le cas contraire, la navbar et la page demandée sont affichées.

```
tsx

// client/src/layouts/ParentLayout.tsx
function ParentLayout() {
  const { auth, setAuth } = useOutletContext<OutletAuthContext>();

  if (auth === null) {
    return <Navigate to="/login" replace />;
  }

  if (auth.role !== "parent") {
    return <Navigate to="/redirection" replace />;
  }

  return (
    <div className={styles.layout}>
      <NavBar />
      <div className={styles.main}>
        <Outlet context={{ auth, setAuth }} />
      </div>
    </div>
  );
}
```

**SchoolLayout** suit la même logique pour le rôle "school".

Cette architecture garantit qu'un parent ne peut jamais accéder aux pages de l'école, et inversement — la protection est assurée au niveau du routeur, avant même que la page ne soit rendue.

## Le composant NavBar

Le composant NavBar s'adapte dynamiquement au rôle de l'utilisateur connecté :

```
tsx

// client/src/components/NavBar/NavBar.tsx (extrait)
function NavBar() {
  const { auth, setAuth } = useOutletContext<OutletAuthContext>();

  const isSchool = auth?.role === "school";
  const styles = isSchool ? schoolStyles : parentStyles;

  const allItems = getNavItems(auth);
  const menuItems = allItems.filter((item) => item.label !== "Déconnexion");

  const displayName =
    auth?.role === "parent"
      ? (auth?.profile as Parent).firstName
      : (auth?.profile as School).name;
  // ...
}
```

Les éléments de navigation sont définis dans un fichier séparé `navItems.ts`, sous forme de tableau typé. Chaque élément contient le chemin, le libellé et l'icône Lucide correspondante. Ce découplage permet de modifier les liens sans toucher au composant `NavBar` lui-même.

**Sur desktop**, la navbar est une barre latérale verticale avec deux états : étendue (icônes + texte) et repliée (icônes seules). Un système de pin/unpin permet à l'utilisateur de figer la navbar en position ouverte ou de la laisser se replier automatiquement au survol :

```
tsx

const [isCollapsed, setIsCollapsed] = useState(false);
const [isPinned, setIsPinned] = useState(false);

const handleMouseEnter = () => {
  if (!isPinned) setIsCollapsed(false);
};

const handleMouseLeave = () => {
  if (!isPinned) setIsCollapsed(true);
};
```

**Sur mobile**, la navbar se transforme en barre horizontale fixe en bas de l'écran, affichant uniquement les icônes — conformément aux conventions UX des applications mobiles modernes.

Deux fichiers CSS Modules distincts — `NavBarSchool.module.css` et `NavBarParent.module.css` — assurent la différenciation visuelle : couleurs bleu foncé pour l'école, touches d'orange pour le parent.

### Accessibilité

La navbar intègre plusieurs attributs d'accessibilité : `aria-label` sur le conteneur aside, `aria-pressed` et `aria-expanded` sur le bouton de pinglage, et `NavLink` avec mise en évidence visuelle de la page active. L'ensemble est navigable au clavier via la touche Tab.

### Impact sur le projet

La création des layouts a constitué un changement structurel pour l'ensemble de l'application. Après leur mise en place, j'ai adapté les styles CSS de toutes les pages et composants existants pour qu'ils fonctionnent correctement dans cette nouvelle architecture — arrière-plans différenciés (dégradé pastel pour parent, gris pour école), marges, paddings et positionnements ajustés.

## Affichage et gestion des tickets côté école (US02)

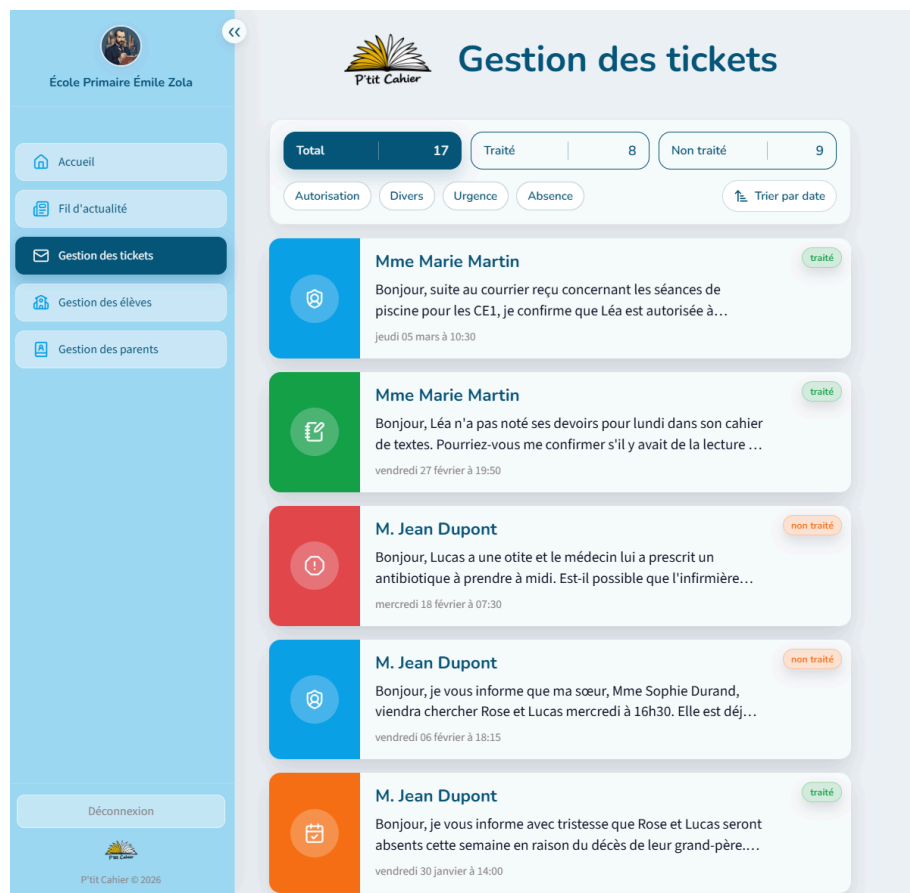
**User story :** En tant qu'école, je souhaite consulter les tickets envoyés par les parents, afin de prendre connaissance de leurs demandes et de les traiter.

### Contexte et enjeu

La user story US02 constitue l'un des deux flux fondamentaux de l'application : elle permet à l'école de consulter les demandes envoyées par les parents. C'est la fonctionnalité que j'ai développée en premier,



et c'est celle qui m'a fait découvrir la chaîne complète du développement full-stack — du composant React à la requête SQL avec jointures multiples.



## Interface utilisateur — la page Tickets

La page Tickets côté école présente la liste des tickets reçus avec un système de filtrage à trois niveaux :

**Filtre par statut** — trois boutons affichant les compteurs dynamiques (Total, Traité, Non traité) :

```
tsx

// client/src/pages/ticket/Tickets.tsx (extrait)
const statusCounts = categoryFilteredTickets.reduce(
  (counts, ticket) => {
    if (ticket.processed) {
      counts.processed += 1;
    } else {
      counts.unprocessed += 1;
    }
    counts.total += 1;
    return counts;
  },
  { total: 0, processed: 0, unprocessed: 0 },
);
```

**Filtre par catégorie** — des boutons générés dynamiquement à partir des catégories présentes dans les tickets, avec sélection multiple (toggle on/off) :

```
tsx

const categories = Array.from(
  new Set(tickets.map((ticket) => ticket.ticketCategoryName)),
);

const toggleCategory = (categoryName: string) => {
  setSelectedCategories((prev) =>
    prev.includes(categoryName)
      ? prev.filter((category) => category !== categoryName)
      : [...prev, categoryName],
  );
};
```

**Tri par date** — un bouton bascule entre l'ordre chronologique et antéchronologique :

```
tsx

const filteredTickets = [...statusFilteredTickets].sort((first, second) => {
  const firstDate = new Date(first.createdAt).getTime();
  const secondDate = new Date(second.createdAt).getTime();
  return sortDirection === "desc"
    ? secondDate - firstDate
    : firstDate - secondDate;
});
```

L'ensemble du filtrage et du tri est réalisé côté client, en mémoire, après le chargement initial des données. Cela offre une réactivité immédiate sans requête supplémentaire au serveur.

### Composant TicketCard

Chaque ticket est rendu par le composant TicketCard, qui affiche un panneau latéral coloré selon la catégorie (rouge pour Urgence, orange pour Absence, vert pour Divers, bleu pour Autorisation), le nom du parent, la catégorie, le contenu tronqué à deux lignes, la date formatée, et un badge de statut (Traité / Non traité).



```
tsx

// client/src/components/TicketCard/TicketCard.tsx (extrait)
<button
  type="button"
  className={` ${styles.card} ${variantClass}`}
  data-type={ticketType}
  data-has-badge={showStatusBadge ? "true" : "false"}
  onClick={onClick}
>
  <div className={styles.leftPanel}>
    <div className={styles.iconCircle}>
      <TicketIcon categoryName={ticket.ticketCategoryName} />
    </div>
  </div>
  <div className={styles.body}>
    <div className={styles.header}>
      <h3 className={styles.nameTitle}>
        {ticket.genre === "F" ? "Mme" : "M."}{ " "}
        {ticket.parentLastName}
      </h3>
      <span className={styles.category}>{ticket.ticketCategoryName}</span>
    </div>
    <p className={styles.content}>{ticket.content}</p>
    <time className={styles.date}>{formattedDate}</time>
  </div>
</button>
```

## Route API et middleware

Le chargement des tickets est déclenché par un appel GET vers `/api/schools/me/tickets`, protégé par les middlewares `verifyToken` et `verifyRole("school")` :

```
typescript

// server/src/router.ts (extrait)
const schoolRouter = express.Router();
schoolRouter.use(authActions.verifyRole("school"));
schoolRouter.get("/me/tickets", ticketActions.browseBySchool);
```

Le contrôleur extrait l'identifiant de l'école depuis le token JWT et le transmet au repository :

```
typescript

// server/src/modules/ticket/ticketActions.ts (extrait)
const browseBySchool: RequestHandler = async (req, res, next) => {
  try {
    const schoolId = Number(req.auth.sub);
    const limit = req.query.limit ? Number(req.query.limit) : undefined;
    const tickets = await ticketRepository.readAllBySchool(schoolId, limit);
    res.json(tickets);
  } catch (err) {
    next(err);
  }
};
```

## Requête SQL — jointures multiples

La requête SQL du repository est l'une des plus complexes du projet. Elle rassemble les données de cinq tables pour construire l'objet ticket complet en une seule requête :

typescript

```
// server/src/modules/ticket/ticketRepository.ts (extrait)
async readAllBySchool(schoolId: number, limit?: number) {
  let sql = `
    SELECT
      t.id,
      t.content,
      t.created_at AS createdAt,
      t.processed AS processed,
      tc.name AS ticketCategoryName,
      p.genre,
      p.first_name AS parentFirstName,
      p.last_name AS parentLastName,
      GROUP_CONCAT(
        CONCAT(s.first_name, ' ', s.last_name)
        ORDER BY s.first_name
        SEPARATOR ', '
      ) AS studentNames
    FROM ticket AS t
    JOIN ticket_category AS tc ON t.ticket_category_id = tc.id
    JOIN parent AS p ON t.parent_id = p.id
    JOIN ticket_student AS tic_stu ON tic_stu.ticket_id = t.id
    JOIN student AS s ON tic_stu.student_id = s.id
    JOIN classroom AS c ON s.classroom_id = c.id
    WHERE c.school_id = ?
    GROUP BY t.id
    ORDER BY t.created_at DESC
  `;
  const sqlParams: (number | string)[] = [schoolId];

  if (limit) {
    sql += " LIMIT ?";
    sqlParams.push(limit);
  }

  const [rows] = await databaseClient.query<Rows>(sql, sqlParams);
  return rows as Ticket[];
}
```

Cette requête illustre plusieurs concepts clés : les jointures entre cinq tables (ticket → ticket\_category → parent → ticket\_student → student → classroom), l'agrégation avec GROUP\_CONCAT pour rassembler les noms des élèves concernés, le filtrage par école via la table classroom (le ticket n'a pas de lien direct avec l'école — il faut remonter via student → classroom → school), les requêtes paramétrées avec placeholders pour la sécurité, et le paramètre LIMIT optionnel pour le tableau de bord.

## Modale de détail d'un ticket (US07)

**User story :** En tant qu'école, je souhaite cliquer sur un ticket pour ouvrir son détail dans une fenêtre modale, puis le fermer pour revenir à la liste, sans rechargement de page.

### Contexte et enjeu

La user story US07 permet à l'école de cliquer sur un ticket dans la liste pour ouvrir son contenu complet dans une fenêtre modale, sans rechargement de page. L'école peut également changer le statut du ticket (traité / non traité) directement depuis cette modale. Cette fonctionnalité est entièrement côté frontend — elle ne nécessite pas de nouvel endpoint API, car les données du ticket sont déjà chargées dans l'état de la page Tickets. Seul le changement de statut déclenche une requête PATCH vers le serveur.



### Le composant TicketModal

La modale est un composant indépendant qui reçoit le ticket sélectionné et une fonction de fermeture en props :

```
// client/src/components/TicketModal/TicketModal.tsx (extrait)
type TicketModalProps = {
  ticket: Ticket;
  onClose: () => void;
  onStatusChange?: (ticketId: number, processed: boolean) => void;
  userRole?: string;
};
```

La modale comporte un overlay (fond assombri), un conteneur de contenu, un bouton de fermeture et, pour le rôle école, un bouton de changement de statut. Le scroll de la page en arrière-plan est bloqué tant que la modale est ouverte. La fermeture peut être déclenchée par trois interactions : clic sur le bouton X, clic sur l'overlay, ou touche Escape au clavier.

### Changement de statut — requête PATCH

Lorsque l'école clique sur le bouton de changement de statut, une requête PATCH est envoyée au serveur:

```
// client/src/pages/ticket/Tickets.tsx (extrait)
const processTicket = async (ticketId: number, processed: boolean) => {
  const updateProcessedStatus = (lastStatus: boolean) => {
    setTickets((prevTickets) =>
      prevTickets.map((ticket) =>
        ticket.id === ticketId
          ? { ...ticket, processed: lastStatus }
          : ticket,
      ),
    );
    setSelectedTicket((prevTicket) =>
      prevTicket && prevTicket.id === ticketId
        ? { ...prevTicket, processed: lastStatus }
        : prevTicket,
    );
  };
};

fetch(
  `${import.meta.env.VITE_API_URL}/api/schools/tickets/${ticketId}/status`,
  {
    method: "PATCH",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${auth?.token}`,
    },
    body: JSON.stringify({ processed }),
  },
)
  .then((response) => response.json())
  .then((ticket) => {
    updateProcessedStatus(ticket.processed);
  })
  .catch((error) => {
    console.error("Update ticket status error:", error);
  });
};
```

L'état est mis à jour simultanément dans la liste des tickets et dans la modale, ce qui garantit la cohérence visuelle sans rechargement.

### Route API et contrôleur côté serveur

```
// server/src/router.ts
schoolRouter.patch("/tickets/:id/status", ticketActions.editStatus);
```

```
// server/src/modules/ticket/ticketActions.ts (extrait)
const editStatus: RequestHandler = async (req, res, next) => {
  try {
    const ticketId = Number(req.params.id);

    if (!Number.isInteger(ticketId) || ticketId <= 0) {
      res.status(StatusCode.BAD_REQUEST).json({
        error: "Identifiant de ticket invalide",
      });
      return;
    }

    if (typeof req.body.processed !== "boolean") {
      res.status(400).json({ error: "processed doit être un boolean" });
      return;
    }

    const wasUpdated = await ticketRepository.updateStatus(
      ticketId,
      req.body.processed,
    );

    if (!wasUpdated) {
      res.status(StatusCode.NOT_FOUND).json({
        error: "Ticket introuvable",
      });
      return;
    }

    res.status(StatusCode.OK).json({ id: ticketId, processed: req.body.processed });
  } catch (err) {
    next(err);
  }
};
```



Ce contrôleur illustre les bonnes pratiques de validation : vérification du type et de la valeur du paramètre d'URL, vérification du type du corps de la requête, et retour d'un statut 404 si le ticket n'existe pas.

### Requête SQL de mise à jour

```
// server/src/modules/ticket/ticketRepository.ts (extrait)
async updateStatus(ticketId: number, processed: boolean) {
  const [result] = await databaseClient.query<Result>(
    "UPDATE ticket SET processed = ? WHERE id = ?",
    [processed ? 1 : 0, ticketId],
  );
  return result.affectedRows > 0;
}
```

La conversion boolean → TINYINT (true → 1, false → 0) est effectuée au niveau du repository, assurant la compatibilité entre le type TypeScript et le type MySQL.

### Accessibilité de la modale

La modale intègre les attributs `role="dialog"` et `aria-modal="true"` pour les lecteurs d'écran. Le focus est placé sur la modale à son ouverture, et la fermeture par la touche `Escape` est implémentée via un écouteur d'événement clavier.

## Déploiement en production sur VPS

**User story :** En tant qu'équipe de développement, nous souhaitons déployer l'application sur un serveur accessible publiquement, afin de permettre une démonstration en conditions réelles devant le jury.

### Contexte et enjeu

Le déploiement de l'application en production a été la partie la plus exigeante de mon travail sur le projet P'tit Cahier. La formation n'avait pratiquement pas abordé le DevOps, et j'ai réalisé l'intégralité de cette mise en production de manière autonome — de la configuration du serveur à la vérification finale dans le navigateur. Cette réalisation ne comporte pas de code applicatif à proprement parler, mais elle mobilise un ensemble de compétences techniques distinctes : administration système Linux, configuration réseau, gestion de base de données en production et documentation.

### Infrastructure

L'application est déployée sur un VPS (Serveur Privé Virtuel) OVHcloud, configuré comme suit :

Système d'exploitation : **Ubuntu 24.04 LTS**. Processeur : 4 vCores. Mémoire : 8 Go RAM. Stockage : 75 Go SSD. Localisation : Gravelines (GRA), France. Adresse IP publique : 51.38.224.29. Nom de domaine : [ptit-cahier.fr](https://ptit-cahier.fr).

## Accès sécurisé au serveur

L'accès au serveur est sécurisé par une paire de clés SSH de type Ed25519. L'authentification par mot de passe a été désactivée, rendant l'accès impossible sans la clé privée correspondante.

Étapes réalisées : génération de la paire de clés Ed25519 en local (`ssh-keygen -t ed25519`), copie de la clé publique sur le serveur dans `~/.ssh/authorized_keys`, configuration des permissions (`chmod 700 .ssh`, `chmod 600 authorized_keys`), désactivation de l'authentification par mot de passe dans la configuration SSH du serveur.

## Procédure de déploiement

Le déploiement initial a suivi les étapes suivantes :

1. Connexion SSH au serveur Ubuntu.
2. Clonage du dépôt GitHub sur le serveur.
3. Installation des dépendances Node.js (`npm install`).
4. Création et configuration de la base de données MySQL.
5. Import du schéma SQL (structure et données initiales).
6. Configuration du fichier `.env` avec les variables d'environnement de production.
7. Build du frontend React (`npm run build --workspace=client`), qui génère les fichiers statiques optimisés dans `client/dist/`.
8. Démarrage du serveur Node.js via PM2.
9. Configuration de Nginx comme reverse proxy.
10. Vérification du bon fonctionnement via le navigateur.

## Configuration de Nginx

Nginx joue un double rôle : il sert les fichiers statiques du frontend React et redirige les requêtes `/api/*` vers le serveur Node.js/Express :

Pour les routes `/api/*` : Nginx effectue un `proxy_pass` vers `http://localhost:3310`, où le serveur Express écoute.

Pour toutes les autres routes : Nginx sert les fichiers statiques du build React situés dans `client/dist/`. Les routes inconnues sont redirigées vers `index.html`, ce qui permet au routeur React côté client de gérer la navigation.

## Gestion du processus avec PM2

PM2 assure la disponibilité permanente de l'application en production : redémarrage automatique en cas de crash, persistance au redémarrage du serveur (`pm2 startup + pm2 save`), monitoring en temps réel (CPU, mémoire, uptime), et accès aux logs de l'application.

## Commandes utilisées en production :

`pm2 status` — vérification de l'état de l'application. `pm2 restart ptitchahier` — redémarrage après une mise à jour. `pm2 logs ptitchahier` — consultation des logs pour le diagnostic.

## Procédure de mise à jour

Chaque mise à jour du code suit une procédure standardisée :

Pour une modification du frontend : `git pull origin main`, `npm run build --workspace=client`, `pm2 restart ptitcahier`.

Pour une modification du backend : `git pull origin main`, `pm2 restart ptitcahier`.

Pour une modification du schéma de base de données : `git pull origin main`, `npm run db:migrate --workspace=server`, `pm2 restart ptitcahier`.

## Sécurité en production

Plusieurs mesures de sécurité ont été mises en place : accès SSH par clé Ed25519 uniquement (mot de passe désactivé), fichier `.env` non versionné contenant les secrets applicatifs, base de données non exposée publiquement (accès localhost uniquement), CORS limité à l'URL du client, mode démonstration (`DEMO_MODE=true`) empêchant la modification des données.

---

## 07 Sécurité de l'application

La sécurité a été traitée comme une préoccupation transversale, intégrée à chaque couche de l'application. Ce chapitre détaille les mesures concrètes mises en place, avec les extraits de code correspondants.

### Hachage des mots de passe — Argon2id

Les mots de passe ne sont jamais stockés en clair dans la base de données. L'algorithme Argon2id a été choisi pour le hachage. Argon2id est le vainqueur du Password Hashing Competition et la recommandation actuelle de l'OWASP. Il est conçu pour résister aux attaques par force brute, aux attaques GPU et aux attaques par canaux auxiliaires (side-channel attacks), grâce à sa consommation élevée de mémoire. Le hachage est réalisé dans un middleware Express dédié, exécuté avant la création du compte utilisateur :

```
// server/src/modules/auth/authActions.ts (extrait)
const hashPassword: RequestHandler = async (req, res, next) => {
  try {
    const { password } = req.body;
    const hashedPassword = await argon2.hash(password, {
      type: argon2.argon2id,
      memoryCost: 19 * 2 ** 10, // ~19 Mo de mémoire
      timeCost: 2, // 2 itérations
      parallelism: 1, // 1 thread
    });
    req.body.hashed_password = hashedPassword;
    req.body.password = undefined; // suppression immédiate du mot de passe en clair
    next();
  } catch (err) {
    next(err);
  }
};
```

Trois points méritent d'être soulignés. Les paramètres (`memoryCost`, `timeCost`, `parallelism`) suivent les recommandations OWASP pour un bon compromis entre sécurité et performance. Le mot de passe en clair est immédiatement supprimé de l'objet requête après hachage (`req.body.password = undefined`), ce qui réduit la surface d'exposition. Le middleware est chaîné dans la route d'inscription, ce qui sépare la responsabilité du hachage de celle de la création du compte :

```
// server/src/router.ts (extrait)
router.post(
  "/register/school",
  schoolActions.validate,
  authActions.hashPassword,
  userActions.add,
  schoolActions.add,
  classroomActions.add,
);
```

## Authentification par JWT

Après vérification des identifiants, le serveur génère un token JWT (JSON Web Token) signé avec une clé secrète stockée en variable d'environnement. Ce token contient l'identifiant du profil utilisateur et son rôle :

```
// server/src/modules/auth/authActions.ts (extrait)
const login: RequestHandler = async (req, res, next) => {
  try {
    // ... validation avec Joi, vérification de l'email ...

    const user = await userRepository.readByEmail(value.email);
    if (!user || user.role !== value.role) {
      res.sendStatus(StatusCodes.UNPROCESSABLE_ENTITY);
      return;
    }

    const verified = await argon2.verify(user.hashed_password, value.password);
    if (!verified) {
      res.sendStatus(StatusCodes.UNPROCESSABLE_ENTITY);
      return;
    }

    const profile =
      user.role === "parent"
        ? await parentRepository.readById(user.id)
        : await schoolRepository.readById(user.id);

    const myPayload: MyPayload = {
      sub: profile.id.toString(),
      role: user.role,
    };

    const token = await jwt.sign(myPayload, process.env.APP_SECRET as string, {
      expiresIn: "1h",
    });

    res.json({ token, profile, role: user.role });
  } catch (err) {
    next(err);
  }
};
```

Plusieurs choix de sécurité sont visibles dans ce code. Le serveur renvoie le même code d'erreur (422 Unprocessable Entity) que l'email soit inconnu, que le rôle ne corresponde pas, ou que le mot de passe soit incorrect — cela empêche un attaquant de distinguer ces cas et donc de deviner quels comptes existent. Le token expire après une heure, limitant la fenêtre d'exploitation en cas de vol. La clé de signature est stockée dans une variable d'environnement, jamais dans le code source.

## Vérification du token sur chaque requête

Chaque requête vers une route protégée passe par le middleware `verifyToken`, qui extrait le token de l'en-tête HTTP et le vérifie :

```
// server/src/modules/auth/authActions.ts (extrait)
const verifyToken: RequestHandler = (req, res, next) => {
  try {
    const authorizationHeader = req.get("Authorization");
    if (authorizationHeader == null) {
      throw new Error("Authorization header is missing");
    }

    const [type, token] = authorizationHeader.split(" ");
    if (type !== "Bearer") {
      throw new Error("Authorization header does not have the 'Bearer' type");
    }

    req.auth = jwt.verify(
      token,
      process.env.APP_SECRET as string
    ) as MyPayload;
    next();
  } catch (err) {
    console.error(err);
    res.sendStatus(StatusCodes.UNAUTHORIZED);
  }
};
```

En cas d'absence de token, de format incorrect ou de signature invalide, le serveur retourne un statut 401 Unauthorized sans révéler la raison exacte du rejet.

## Contrôle des rôles

Au-delà de l'authentification, le middleware `verifyRole` vérifie que le rôle contenu dans le token correspond à la ressource demandée :

```
typescript

// server/src/modules/auth/authActions.ts (extrait)
const verifyRole = (role: "parent" | "school"): RequestHandler => {
  return (req, res, next) => {
    if (!req.auth) {
      res.sendStatus(StatusCode.UNAUTHORIZED);
      return;
    }
    if (req.auth.role !== role) {
      res.sendStatus(StatusCode.FORBIDDEN);
      return;
    }
    next();
  };
};
```

Les routes parent et école sont séparées dans des sous-routeurs dédiés, chacun protégé par son propre middleware :

```
typescript

// server/src/router.ts (extrait)
router.use(authActions.verifyToken);

const parentRouter = express.Router();
parentRouter.use(authActions.verifyRole("parent"));

const schoolRouter = express.Router();
schoolRouter.use(authActions.verifyRole("school"));

router.use("/parents", parentRouter);
router.use("/schools", schoolRouter);
```

Cette architecture garantit qu'un parent authentifié ne peut pas accéder aux routes de l'école (statut 403 Forbidden), et inversement.

## Vérification d'appartenance des données

La protection par rôle ne suffit pas à elle seule. Un parent authentifié pourrait tenter de créer un ticket pour l'enfant d'un autre parent. Le middleware de validation vérifie donc que chaque donnée manipulée appartient bien à l'utilisateur connecté :

typescript

```
// server/src/modules/ticket/ticketActions.ts (extrait)
const validate: RequestHandler = async (req, res, next) => {
  try {
    // ... validation de la structure avec Joi ...

    const parentId = Number(req.auth.sub);
    const studentIds = value.studentIds;

    for (const studentId of studentIds) {
      const currentStudent = await studentRepository.read(studentId);
      if (!currentStudent) {
        res.status(StatusCode.NOT_FOUND)
          .json({ error: "Étudiant introuvable" });
        return;
      }
      if (currentStudent.parentId !== parentId) {
        res.status(StatusCode.UNPROCESSABLE_ENTITY)
          .json({ error: "L'étudiant n'appartient pas à ce parent" });
        return;
      }
    }

    req.body = value;
    next();
  } catch (err) {
    next(err);
  }
};
```

La même logique est appliquée côté école : une école ne peut modifier ou supprimer que ses propres annonces, vérifié via le schoolId extrait du token.

## Validation des données entrantes — Joi

Chaque requête POST, PUT et PATCH passe par un middleware de validation Joi avant tout traitement. Joi vérifie la structure, le type, la longueur et les contraintes de chaque champ :

```
typescript

// server/src/modules/ticket/ticketActions.ts (extrait)
const newTicket = joi.object({
  content: joi.string().max(1000).required(),
  ticketCategoryId: joi.number().integer().positive().required(),
  studentIds: joi
    .array()
    .items(joi.number().integer().positive())
    .min(1)
    .required(),
});

const { error, value } = newTicket.validate(req.body);
if (error) {
  res.status(StatusCode.BAD_REQUEST)
    .json({ error: "Les données envoyées sont invalides" });
  return;
}
```

Les données invalides sont rejetées avec un statut 400 Bad Request avant d'atteindre le contrôleur ou la base de données. Le message d'erreur est générique (pas de détail sur quel champ a échoué), pour ne pas guider un attaquant.

## Protection contre les injections SQL

Toutes les requêtes SQL de l'application utilisent des placeholders (?) avec des paramètres passés séparément au driver mysql2. Aucune requête n'est construite par concaténation de chaînes :

```
typescript

// Exemple : requête paramétrée
const [rows] = await databaseClient.query<Rows>(
  "SELECT * FROM user WHERE email = ? LIMIT 1",
  [email],
);
```

Le driver mysql2 échappe automatiquement les valeurs des paramètres, ce qui rend toute injection SQL impossible, quelle que soit la valeur fournie par l'utilisateur.



## Configuration CORS

La configuration CORS limite strictement les origines autorisées à l'URL du client, définie en variable d'environnement :

```
typescript

// server/src/app.ts (extrait)
import cors from "cors";

if (process.env.CLIENT_URL != null) {
  app.use(cors({ origin: [process.env.CLIENT_URL] }));
}
```

L'utilisation de `cors()` sans paramètre, qui autoriserait toutes les origines, a été explicitement évitée. Le commentaire dans le code source le mentionne : cette configuration ouverte « peut poser des problèmes de sécurité ».

## Gestion des secrets

Les informations sensibles sont stockées dans un fichier `.env` situé sur le serveur, non versionné dans le dépôt Git :

Paramètres de connexion MySQL (hôte, port, utilisateur, mot de passe, nom de la base). Port d'écoute du serveur applicatif. URL du client pour la configuration CORS. Clé secrète pour la signature des tokens JWT. Indicateurs de mode démonstration et lecture seule.

Un fichier `.env.sample` sans valeurs sensibles est fourni dans le dépôt comme modèle, permettant à un nouveau développeur de configurer son environnement sans accéder aux secrets de production.

## Protection des données en production

L'application déployée fonctionne en mode démonstration avec deux variables d'environnement :

**DEMO\_MODE=true** — les fonctionnalités de modification des données sont désactivées, empêchant toute altération du contenu lors de la présentation devant le jury.

**READ\_ONLY\_MODE=true** — couche de protection supplémentaire garantissant l'intégrité des données de démonstration.

## Synthèse des mesures de sécurité

| Menace                     | Mesure                      | Implémentation                            |
|----------------------------|-----------------------------|-------------------------------------------|
| Vol de mot de passe        | Hachage Argon2id            | Middleware hashPassword                   |
| Accès non autorisé         | Authentification JWT        | Middleware verifyToken                    |
| Élévation de privilèges    | Contrôle des rôles          | Middleware verifyRole + sous-routeurs     |
| Accès aux données d'autrui | Vérification d'appartenance | Middleware validate (parentId, schoolId)  |
| Données malformées         | Validation Joi              | Middleware validate avant chaque écriture |
| Injection SQL              | Requêtes paramétrées        | Placeholders ? dans mysql2                |
| Requêtes cross-origin      | CORS restrictif             | Origine limitée à CLIENT_URL              |
| Secrets exposés            | Variables d'environnement   | Fichier .env non versionné                |
| Altération en démo         | Mode lecture seule          | DEMO_MODE + READ_ONLY_MODE                |

## 08 Jeu d'essai

Ce chapitre présente le test fonctionnel complet de la fonctionnalité d'authentification (connexion à l'application). Cette fonctionnalité a été choisie car elle constitue le point d'entrée de l'ensemble de l'application et mobilise simultanément le frontend (formulaire, validation côté client, gestion de l'état), le backend (route API, validation Joi, vérification en base, génération de token JWT) et la sécurité (hachage Argon2id, réponses non discriminantes).

Pour chaque scénario, sont documentés : les données d'entrée (action de l'utilisateur), le résultat attendu (comportement prévu par la spécification), le résultat obtenu (comportement réellement observé), et l'analyse des éventuels écarts.

### Environnement de test

Application déployée en production : <https://ptit-cahier.fr>. Navigateur : Chrome 131 (dernière version stable). Comptes de démonstration utilisés :

Parent — email : example@parent1.com, mot de passe : Password123, rôle : parent. École — email : example@school1.com, mot de passe : Password123, rôle : school.

### Scénario 1 — Connexion réussie (rôle parent)

**Données d'entrée :** l'utilisateur saisit l'email example@parent1.com, le mot de passe Password123, sélectionne le rôle Parent et clique sur le bouton Connexion.

**Résultat attendu :** le serveur retourne un token JWT et le profil utilisateur. Le client stocke ces données dans le localStorage, puis redirige l'utilisateur vers /parent/home (tableau de bord parent). La navbar parent est affichée.

**Résultat obtenu :** comportement conforme. La redirection vers /parent/home s'effectue sans délai perceptible. Le tableau de bord affiche le nom de l'école, le carrousel des dernières annonces et la liste des tickets récents. La navbar parent est visible avec les liens Accueil, Fil d'actualité, Demande à l'école.

**Écart :** aucun.



## Scénario 2 — Connexion réussie (rôle école)

**Données d'entrée :** l'utilisateur saisit l'email example@school1.com, le mot de passe password, sélectionne le rôle École et clique sur Connexion.

**Résultat attendu :** redirection vers /school/home (tableau de bord école). La navbar école est affichée avec les liens spécifiques au rôle école.

**Résultat obtenu :** comportement conforme. Le tableau de bord affiche les indicateurs clés (nombre d'annonces, d'élèves, de classes, de parents) et la liste des derniers tickets reçus. La navbar école est visible avec les liens Accueil, Fil d'actualité, Tickets, Élèves, Parents.

**Écart :** aucun.

## Scénario 3 — Mot de passe incorrect

**Données d'entrée :** l'utilisateur saisit l'email example@parent1.com, le mot de passe mauvais\_mot\_de\_passe, sélectionne le rôle Parent et clique sur Connexion.

**Résultat attendu :** le serveur retourne un statut 422 (Unprocessable Entity). L'interface affiche un message d'erreur. L'utilisateur reste sur la page de connexion. Aucun token n'est stocké.

**Résultat obtenu :** comportement conforme. Un message d'erreur générique s'affiche (sans préciser que c'est le mot de passe qui est incorrect — mesure de sécurité). L'utilisateur reste sur /login. Le localStorage ne contient aucun token.

**Écart :** aucun.

#### Scénario 4 — Email inexistant

**Données d'entrée :** l'utilisateur saisit l'email inexistant@test.com, le mot de passe password, sélectionne le rôle Parent et clique sur Connexion.

**Résultat attendu :** le serveur retourne un statut 422. Même message d'erreur que pour un mot de passe incorrect (réponse non discriminante pour empêcher l'énumération des comptes existants).

**Résultat obtenu :** comportement conforme. Le message d'erreur affiché est identique à celui du scénario 3. Un attaquant ne peut pas distinguer un email inexistant d'un mot de passe incorrect.

**Écart :** aucun.

#### Scénario 5 — Rôle ne correspondant pas au compte

**Données d'entrée :** l'utilisateur saisit l'email example@parent1.com (compte parent), le mot de passe password, mais sélectionne le rôle École et clique sur Connexion.

**Résultat attendu :** le serveur rejette la demande avec un statut 422, car le rôle sélectionné ne correspond pas au rôle enregistré en base de données.

**Résultat obtenu :** comportement conforme. Message d'erreur identique aux scénarios 3 et 4. L'utilisateur ne peut pas déterminer si l'erreur vient du rôle, du mot de passe ou de l'email.

**Écart :** aucun.

#### Scénario 6 — Champs vides

**Données d'entrée :** l'utilisateur laisse le champ email vide et clique sur Connexion.

**Résultat attendu :** la validation empêche l'envoi de la requête. Un message d'erreur invite l'utilisateur à remplir les champs obligatoires.

**Résultat obtenu :** comportement conforme. La validation HTML native (attribut required sur les champs input) empêche la soumission du formulaire. Le navigateur affiche un message indiquant que le champ est requis. Aucune requête n'est envoyée au serveur.

**Écart :** aucun.

#### Scénario 7 — Format d'email invalide

**Données d'entrée :** l'utilisateur saisit abc (sans @, sans domaine), le mot de passe password, sélectionne le rôle Parent et clique sur Connexion.

**Résultat attendu :** la validation empêche l'envoi. Côté serveur, si la requête parvenait malgré tout, Joi rejeterait l'email avec un statut 400.

**Résultat obtenu :** comportement conforme. La validation HTML native (input type="email") détecte le format invalide et bloque la soumission. Aucune requête n'est envoyée au serveur.

**Écart :** aucun.

### Scénario 8 — Tentative d'injection SQL dans le champ email

**Données d'entrée :** l'utilisateur saisit dans le champ email la chaîne ' OR 1=1 --, le mot de passe password, sélectionne le rôle Parent et clique sur Connexion.

**Résultat attendu :** l'injection SQL échoue grâce aux requêtes paramétrées. Le serveur traite la chaîne comme une valeur littérale, ne trouve aucun utilisateur correspondant, et retourne un statut 422.

**Résultat obtenu :** comportement conforme. La requête SQL paramétrée (`SELECT * FROM user WHERE email = ? LIMIT 1`) traite la chaîne d'injection comme une simple valeur de paramètre. Aucun utilisateur n'est trouvé. Le serveur retourne 422. La base de données n'est pas affectée.

**Écart :** aucun.

### Scénario 9 — Accès à une route protégée sans authentification

**Données d'entrée :** l'utilisateur tape directement l'URL /parent/home dans la barre d'adresse du navigateur, sans être connecté.

**Résultat attendu :** le ParentLayout détecte l'absence de token et redirige automatiquement vers /login.

**Résultat obtenu :** comportement conforme. La redirection vers /login est immédiate. Aucune donnée de la page /parent/home n'est affichée, même brièvement.

**Écart :** aucun.

### Scénario 10 — Accès à une route du mauvais rôle

**Données d'entrée :** un utilisateur connecté en tant que parent tape directement l'URL /school/tickets dans la barre d'adresse.

**Résultat attendu :** le SchoolLayout détecte que le rôle du token est "parent" et non "school", et redirige vers /redirection.

**Résultat obtenu :** comportement conforme. L'utilisateur est redirigé vers /redirection, qui le renvoie ensuite vers son propre tableau de bord (/parent/home). Aucune donnée de la page école n'est accessible. Écart : aucun.

## Analyse

L'ensemble des dix scénarios testés produit des résultats conformes aux attentes. Aucun écart n'a été identifié.

Les trois premiers scénarios valident le flux nominal de connexion. Les scénarios 3 à 5 vérifient que les réponses d'erreur sont non discriminantes — un attaquant ne peut pas distinguer un email inexistant d'un mot de passe incorrect ou d'un mauvais rôle, ce qui empêche l'énumération des comptes. Les scénarios 6 et 7 confirment la double validation (côté client par les attributs HTML, côté serveur par Joi). Le scénario 8 vérifie la résistance aux injections SQL. Les scénarios 9 et 10 testent la protection des routes côté frontend (layouts avec vérification de token et de rôle).

La sécurité de l'authentification repose sur trois niveaux de défense : la validation côté client (empêche les envois accidentels de données malformées), la validation côté serveur avec Joi (rejette les données invalides avant tout traitement), et la vérification en base de données avec requêtes paramétrées (empêche les injections et vérifie les identifiants).

---

## 09 Conclusion

### Compétences acquises

Le projet P'tit Cahier m'a permis de mobiliser et de consolider l'ensemble des compétences du référentiel DWWWM, en les appliquant à un projet concret, fonctionnel et déployé en production.

Côté frontend, j'ai approfondi ma maîtrise de React et TypeScript en développant des composants dynamiques avec gestion d'état, filtrage combiné, modales et navigation conditionnelle par rôle. La création des layouts m'a confrontée à une vraie décision architecturale — proposer une solution, la défendre devant l'équipe et l'implémenter — ce qui dépasse le cadre de l'exécution technique.

Côté backend, j'ai appris à construire une API REST structurée avec Express, à écrire des requêtes SQL complexes avec jointures multiples et agrégations, et à implémenter une chaîne de sécurité complète : hachage Argon2id, authentification JWT, contrôle des rôles par middleware, validation Joi et requêtes paramétrées.

Le déploiement sur VPS a été l'apprentissage le plus exigeant et le plus formateur. En partant de zéro, sans base acquise en formation, j'ai configuré un serveur Linux, mis en place Nginx comme reverse proxy, géré un processus Node.js avec PM2 et sécurisé l'accès par clé SSH. Cette expérience m'a donné une vision complète du cycle de vie d'une application web, de la première ligne de code à sa mise à disposition publique.

Enfin, le travail sur l'accessibilité numérique — choix typographiques, contrastes, attributs ARIA, navigation au clavier — a confirmé mon intérêt profond pour ce domaine. C'est une dimension que je

souhaite approfondir et qui constitue la direction dans laquelle je veux développer ma spécialité professionnelle.

## **Retour d'expérience**

Ce projet m'a appris autant sur le plan humain que technique. Travailler sur un sujet que je ne connaissais pas — la communication scolaire — m'a obligée à développer une compétence que le code seul ne donne pas : la capacité à comprendre un domaine métier par l'écoute et le questionnement. Écouter mes collègues parents d'élèves, interroger mes amis, observer les usages réels — c'est cette démarche qui a permis de créer des données de démonstration crédibles et une application qui parle à son public cible.

Le passage de six à quatre membres en cours de projet a été un défi organisationnel. Il m'a appris à reprioriser, à accepter que certaines fonctionnalités ne seraient pas livrées, et à concentrer l'effort sur ce qui compte le plus — un produit fonctionnel, déployé et démontrable, plutôt qu'un produit ambitieux mais inachevé.

## **Perspectives d'évolution**

P'tit Cahier est un produit fonctionnel, mais plusieurs améliorations le rapprocheraient d'une application prête pour un usage réel :

**HTTPS avec certificat SSL** — actuellement l'application est servie en HTTP. L'ajout d'un certificat Let's Encrypt via Certbot sécuriserait les échanges entre le navigateur et le serveur, ce qui est indispensable pour une application manipulant des données personnelles.

**Notifications en temps réel** — l'intégration de WebSocket (via Socket.io) permettrait aux parents de recevoir une notification instantanée lorsqu'une nouvelle annonce est publiée, sans avoir à rafraîchir la page.

**Agenda personnalisé** — une page calendrier affichant les événements à venir (sorties scolaires, spectacles, réunions) avec des pastilles visuelles sur les dates concernées, telle que prévue dans la user story US16.

**Confirmation de lecture** — un système d'accusé de réception permettant à l'école de savoir quels parents ont lu une annonce, particulièrement utile pour les communications administratives importantes.

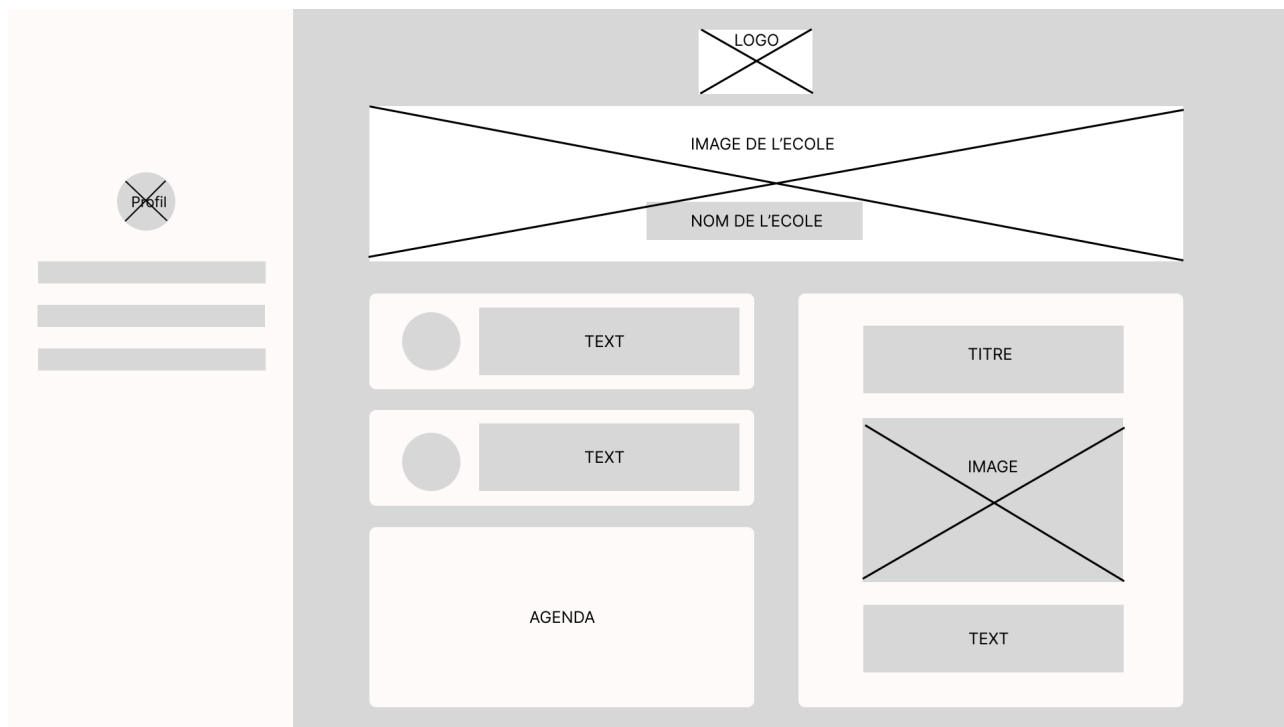
**Pipeline CI/CD** — l'automatisation du déploiement via GitHub Actions remplacerait la procédure manuelle actuelle (git pull, build, pm2 restart) et réduirait le risque d'erreur humaine lors des mises à jour.

**Application mobile** — une adaptation avec React Native ou une Progressive Web App (PWA) offrirait une expérience native sur smartphone, avec notifications push et accès hors ligne.

## Annexes

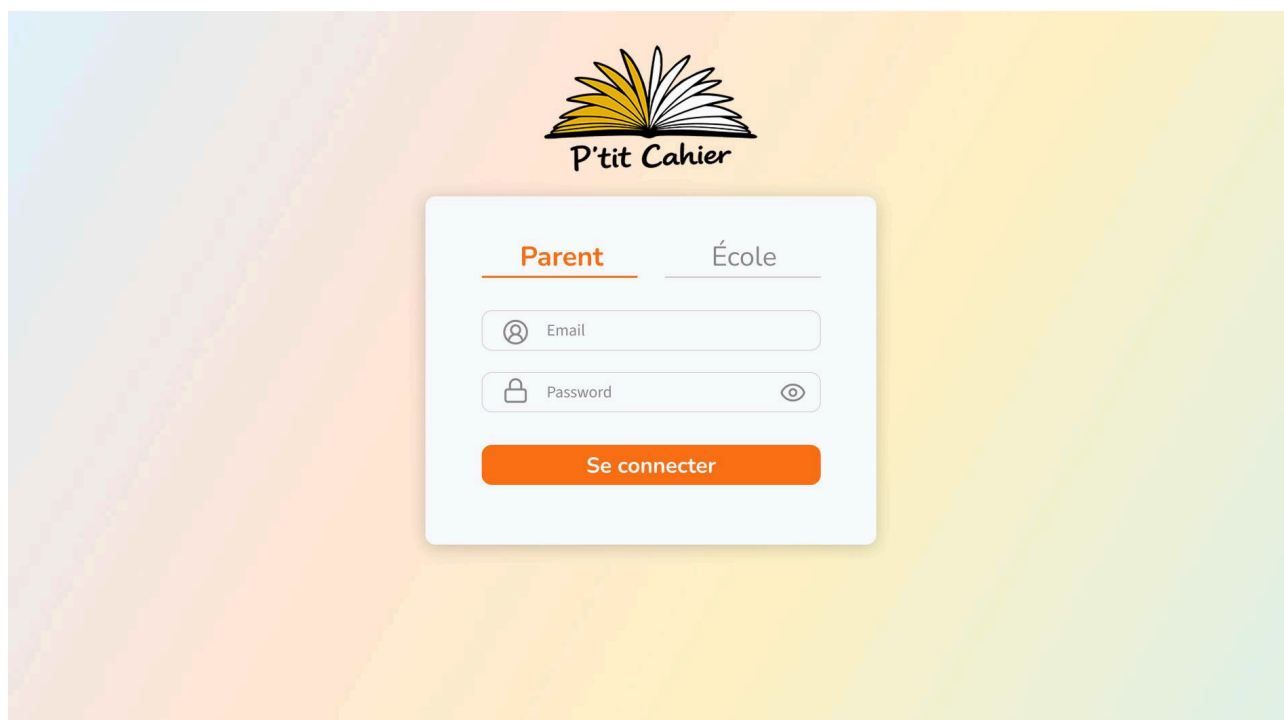
### Annexe A — Wireframes

Wireframes basse fidélité réalisés en phase de conception, présentant la structure et la navigation de l'application en versions desktop et mobile.



### Annexe B — Maquettes haute fidélité

Maquettes UI réalisées dans Figma, intégrant la charte graphique, les composants visuels et les états d'interface.





Total 9

Traité 4

Non-traité 5

Absence

Authorisation

Urgence

Divers



Nom du Parent

Traité

Lucas sera absent ce mercredi 12 Janvier, jusqu'au 15 inclus. Je raconte des trucs super interessant ! Blabliblablabl blabli  
En vous souhaitanta blablabla.

Lundi 10 janvier à 08h56



Nom du Parent

Non Traité

Lucas sera absent ce mercredi 12 Janvier, jusqu'au 15 inclus. Je raconte des trucs super interessant ! Blabliblablabl bliablia  
En vous souhaitanta blablabla.

Lundi 10 janvier à 08h56



Nom du Parent

Non Traité

Lucas sera absent ce mercredi 12 Janvier, jusqu'au 15 inclus. Je raconte des trucs super interessant ! Blabliblablabl bliablia  
En vous souhaitanta blablabla.

Lundi 10 janvier à 08h56

Annexe C — Charte graphique

Palette de couleurs, typographie et composants visuels définis pour le projet P'tit Cahier.



Fonts

**Nunito Bold - Primary title**

Nunito SemiBold - Card Title

Sans Source 3 - Text

Backgrounds

Parent



linear gradient

School



linear gradient

Card



#F8FAFC

Card

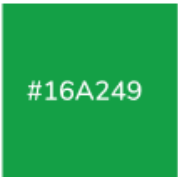


#EEF2F7

Icons & accents



#6D5BD0



#16A249



#F97015



#E5484D



#0da2e7

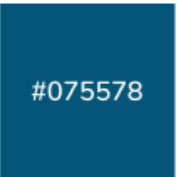


#e7b008

Text



#0F1729



#075578



#898888

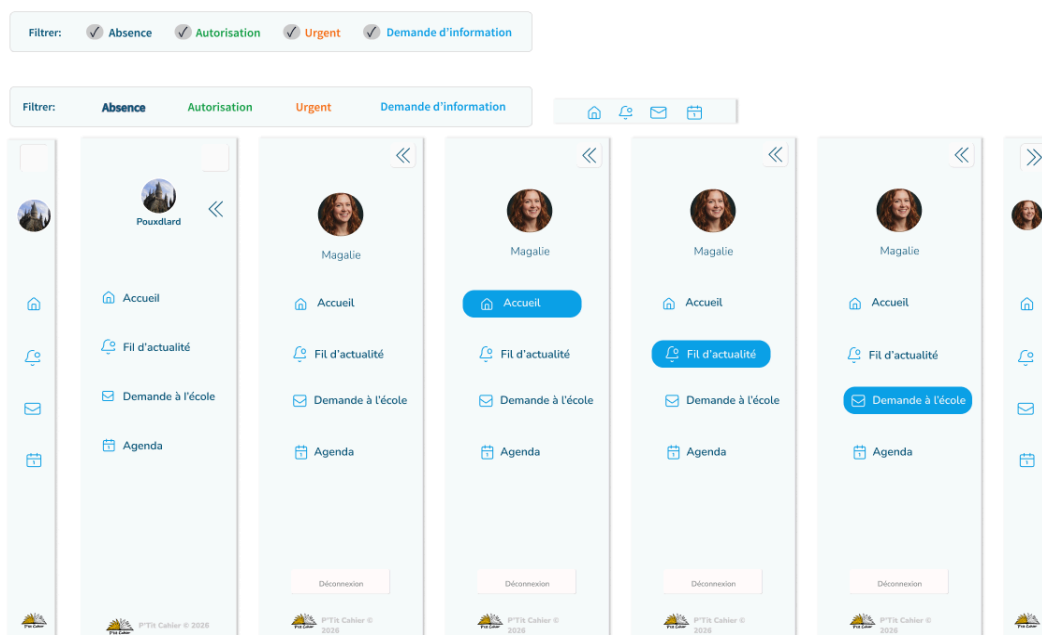


#F8FAFC

Button



Button



## Annexe D — Schéma SQL

Script de création de la base de données MySQL (structure des tables, contraintes et données de référence).

La table `announcement` contient une colonne `classroom_id` (nullable) qui n'est pas exploitée dans la version actuelle de l'application. Le ciblage des annonces passe exclusivement par la table de liaison `announcement_student`, qui offre une granularité plus fine en permettant d'adresser une annonce à des élèves individuels, indépendamment de leur classe. La colonne `classroom_id` a été conservée dans le schéma en vue d'une évolution future permettant un ciblage direct par classe.

### Tables d'authentification

```
CREATE TABLE user (
  id      INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  email   VARCHAR(255) NOT NULL UNIQUE,
  hashed_password VARCHAR(255) NOT NULL,
  role    ENUM('parent', 'school') NOT NULL
);

CREATE TABLE school (
  id      INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  name    VARCHAR(120) NOT NULL,
  photo_url VARCHAR(255) NULL,
  user_id INT UNSIGNED NOT NULL UNIQUE,
  FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE
);
```

```
CREATE TABLE parent (
  id      INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  last_name VARCHAR(120) NOT NULL,
  first_name VARCHAR(120) NOT NULL,
  genre   ENUM('M', 'F') NOT NULL,
  photo_url VARCHAR(255) NULL,
  user_id INT UNSIGNED NOT NULL UNIQUE,
  FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE
);
```

## Tables de référence (categories)

```
CREATE TABLE announcement_category (
  id      INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  name    VARCHAR(120) NOT NULL UNIQUE,
  description VARCHAR(120),
  color   VARCHAR(6) NOT NULL,
  icon    VARCHAR(20) NOT NULL
);
```

```
CREATE TABLE ticket_category (
  id      INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  name    VARCHAR(120) NOT NULL UNIQUE,
  description VARCHAR(120),
  color   VARCHAR(6) NOT NULL,
  icon    VARCHAR(20) NOT NULL
);
```

## Structure scolaire

```
CREATE TABLE classroom (
  id      INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  name    VARCHAR(100) NOT NULL,
  school_id INT UNSIGNED NOT NULL,
  FOREIGN KEY (school_id) REFERENCES school(id)
);
```

```
CREATE TABLE student (
  id      INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  last_name VARCHAR(120) NOT NULL,
  first_name VARCHAR(120) NOT NULL,
  classroom_id INT UNSIGNED NOT NULL,
  parent_id INT UNSIGNED NULL,
  FOREIGN KEY (classroom_id) REFERENCES classroom(id),
  FOREIGN KEY (parent_id) REFERENCES parent(id) ON DELETE SET NULL
);
```

## Announces

```
CREATE TABLE announcement (
  id      INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  title   VARCHAR(120) NOT NULL,
  content VARCHAR(1000) NOT NULL,
  image_url VARCHAR(255) NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  announcement_category_id INT UNSIGNED NOT NULL,
  school_id INT UNSIGNED NOT NULL,
  classroom_id INT UNSIGNED NULL,
  FOREIGN KEY (announcement_category_id) REFERENCES announcement_category(id),
  FOREIGN KEY (school_id) REFERENCES school(id),
  FOREIGN KEY (classroom_id) REFERENCES classroom(id)
);

CREATE TABLE announcement_student (
  announcement_id INT UNSIGNED NOT NULL,
  student_id INT UNSIGNED NOT NULL,
  PRIMARY KEY (announcement_id, student_id),
  FOREIGN KEY (announcement_id) REFERENCES announcement(id) ON DELETE CASCADE,
  FOREIGN KEY (student_id) REFERENCES student(id) ON DELETE CASCADE
);
```

## Tickets

```
CREATE TABLE ticket (
  id      INT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  title   VARCHAR(120) NOT NULL DEFAULT 'Sans titre',
  content VARCHAR(1000) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  processed TINYINT(1) NOT NULL DEFAULT 0,
  parent_id INT UNSIGNED NULL,
  ticket_category_id INT UNSIGNED NOT NULL,
  FOREIGN KEY (parent_id) REFERENCES parent(id) ON DELETE SET NULL,
  FOREIGN KEY (ticket_category_id) REFERENCES ticket_category(id)
);

CREATE TABLE ticket_student (
  ticket_id INT UNSIGNED NOT NULL,
  student_id INT UNSIGNED NOT NULL,
  PRIMARY KEY (ticket_id, student_id),
  FOREIGN KEY (ticket_id) REFERENCES ticket(id) ON DELETE CASCADE,
  FOREIGN KEY (student_id) REFERENCES student(id) ON DELETE CASCADE
);
```

## Donnees de reference — Categories d'annonces

INSERT INTO announcement\_category (id, name, description, color, icon)

VALUES

(1, "Vie de l'ecole",  
"Actualites et moments de vie de l'ecole",  
"6d5bd0", "School"),

(2, "Administratif",  
"Communications administratives",  
"16a249", "ClipboardList"),

(3, "Evenement",  
"Evenements et temps forts a venir",  
"0da2e7", "CalendarDays");

## Donnees de reference — Categories de tickets

INSERT INTO ticket\_category (id, name, description, color, icon)

VALUES

(1, "Urgence",  
"Situation necessitant une action rapide",  
"e5484d", "OctagonAlert"),

(2, "Absence",  
"Signaler une absence prevue ou imprevue",  
"f97015", "CalendarCheck"),

(3, "Divers",  
"Question ou renseignement",  
"16a249", "NotebookPen"),

(4, "Autorisation",  
"Demander une permission ou un accord",  
"0da2e7", "ShieldUser");

---

## Annexe E — Liens et accès

Application en ligne : <https://ptit-cahier.fr> Dépôt GitHub : <https://github.com/Dreamoire/ptitcahier>

Compte de démonstration parent : example@parent1.com / Password123

Compte de démonstration école : example@school1.com / Password123